Red Hat OpenShift AI Self-Managed 3.4

Evaluating AI systems

Evaluate your OpenShift AI models for accuracy, relevance, and consistency

Last Updated: 2026-05-15

Red Hat OpenShift AI Self-Managed 3.4 Evaluating AI systems

Evaluate your OpenShift AI models for accuracy, relevance, and consistency

Legal Notice

Abstract

Evaluate your OpenShift AI models for accuracy, relevance, and consistency.

Table of Contents

CHAPTER 1. OVERVIEW OF EVALUATING AI SYSTEMS

Evaluate your AI systems to generate an analysis of your model's ability by using the following TrustyAI tools:

- **LM-Eval**: You can use TrustyAI to monitor your LLM against a range of different evaluation tasks and to ensure the accuracy and quality of its output. Features such as summarization, language toxicity, and question-answering accuracy are assessed to inform and improve your model parameters.
- **RAGAS**: Use Retrieval-Augmented Generation Assessment (RAGAS) with TrustyAI to measure and improve the quality of your RAG systems in OpenShift AI. RAGAS provides objective metrics that assess retrieval quality, answer relevance, and factual consistency.
- **Llama Stack**: Use Llama Stack components and providers with TrustyAI to evaluate and work with LLMs.

CHAPTER 2. EVALUATING LARGE LANGUAGE MODELS

A large language model (LLM) is a type of artificial intelligence (AI) program that is designed for natural language processing tasks, such as recognizing and generating text.

As a data scientist, you might want to monitor your large language models against a range of metrics, in order to ensure the accuracy and quality of its output. Features such as summarization, language toxicity, and question-answering accuracy can be assessed to inform and improve your model parameters.

Red Hat OpenShift AI now offers Language Model Evaluation as a Service (LM-Eval-aaS), in a feature called LM-Eval. LM-Eval provides a unified framework to test generative language models on a vast range of different evaluation tasks.

The following sections show you how to create an **LMEvalJob** custom resource (CR) which allows you to activate an evaluation job and generate an analysis of your model's ability.

2.1. SETTING UP LM-EVAL

LM-Eval is a service designed for evaluating large language models that has been integrated into the TrustyAI Operator.

The service is built on top of two open-source projects:

- LM Evaluation Harness, developed by EleutherAI, that provides a comprehensive framework for evaluating language models
- Unitxt, a tool that enhances the evaluation process with additional functionalities

The following information explains how to create an **LMEvalJob** custom resource (CR) to initiate an evaluation job and get the results.

Global settings for LM-Eval

Configurable global settings for LM-Eval services are stored in the TrustyAI operator global **ConfigMap**, named **trustyai-service-operator-config**. The global settings are located in the same namespace as the operator.

You can configure the following properties for LM-Eval:

Table 2.1. LM-Eval properties

Property	Default	Description
lmes-detect-device	**true/false**	Detect if there are GPUs available and assign a value for the **--device argument** for LM Evaluation Harness. If GPUs are available, the value is **cuda**. If there are no GPUs available, the value is**cpu**.
lmes-pod-image	**quay.io/trustyai/ta-lmes-job:latest**	The image for the LM-Eval job. The image contains the Python packages for LM Evaluation Harness and Unitxt.

Property	Default	Description
lmes-driver-image	**quay.io/trustyai/ta-lmes-driver:latest**	The image for the LM-Eval driver. For detailed information about the driver, see the **cmd/lmes_driver** directory.
lmes-image-pull-policy	**Always**	The image-pulling policy when running the evaluation job.
lmes-default-batch-size	8	The default batch size when invoking the model inference API. Default batch size is only available for local models.
lmes-max-batch-size	24	The maximum batch size that users can specify in an evaluation job.
lmes-pod-checking-interval	10s	The interval to check the job pod for an evaluation job.

After updating the settings in the **ConfigMap**, restart the operator to apply the new values.

2.2. ENABLING EXTERNAL RESOURCE ACCESS FOR LMEVAL JOBS

LMEval jobs do not allow internet access or remote code execution by default. When configuring an **LMEvalJob**, it may require access to external resources, for example task datasets and model tokenizers, usually hosted on Hugging Face. If you trust the source and have reviewed the content of these artifacts, an **LMEvalJob** can be configured to automatically download them.

Follow the steps below to enable online access and remote code execution for LMEval jobs. Choose to update these settings by using either the CLI or in the console. Enable one or both settings according to your needs.

2.2.1. Enabling online access and remote code execution for LMEval Jobs using the CLI

You can enable online access using the CLI for LMEval jobs by setting the **allowOnline** specification to **true** in the **LMEvalJob** custom resource (CR). You can also enable remote code execution by setting the **allowCodeExecution** specification to **true**. Both modes can be used at the same time.

IMPORTANT

Enabling online access or code execution involves a security risk. Only use these configurations if you trust the source(s).

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have downloaded and installed the OpenShift AI command-line interface (CLI). See Installing the OpenShift CLI.

Procedure

1. Get the current **DataScienceCluster** resource, which is located in the **redhat-ods-operator** namespace:

```
$ oc get datasciencecluster -n redhat-ods-operator
```

Example output

```
NAME          AGE
default-dsc   10d
```

2. Enable online access and code execution for the cluster in the **DataScienceCluster** resource with the **permitOnline** and **permitCodeExecution** specifications. For example, create a file named **allow-online-code-exec-dsc.yaml** with the following contents:

Example allow-online-code-exec-dsc.yaml resource enabling online access and remote code execution

```
apiVersion: datasciencecluster.opendatahub.io/v2
kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
# ...
  components:
    trustyai:
      managementState: Managed
      eval:
        lmeval:
          permitOnline: allow
          permitCodeExecution: allow
# ...
```

The **permitCodeExecution** and **permitOnline** settings are disabled by default with a value of **deny**. You must explicitly enable these settings in the **DataScienceCluster** resource for the **LMEvalJob** instance to enable internet access or permission to run any externally downloaded code.

3. Apply the updated **DataScienceCluster**:

```
$ oc apply -f allow-online-code-exec-dsc.yaml -n redhat-ods-operator
```

   a. Optional: Run the following command to check that the **DataScienceCluster** is in a healthy state:

```
$ oc get datasciencecluster default-dsc
```

Example output

```
NAME         READY   REASON
default-dsc  True
```

4. For new LMEval jobs, define the job in a YAML file as shown in the following example. This configuration requests both internet access, with **allowOnline: true**, and permission for remote code execution with, **allowCodeExecution: true**:

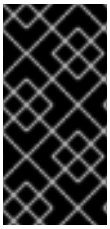
Example lmevaljob-with-online-code-exec.yaml

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: lmevaljob-with-online-code-exec
  namespace: <your_namespace>
spec:
# ...
  allowOnline: true
  allowCodeExecution: true
# ...
```

The **allowOnline** and **allowCodeExecution** settings are disabled by default with a value of **false** in the **LMEvalJob** CR.

5. Deploy the LMEval Job:

```
$ oc apply -f lmevaljob-with-online-code-exec.yaml -n <your_namespace>
```



IMPORTANT

If you upgrade to version 2.25, some TrustyAI **LMEvalJob** CR configuration values might be overwritten. The new deployment prioritizes the value on the 2.25 version **DataScienceCluster**. Existing LMEval jobs are unaffected. Verify that all **DataScienceCluster** values are explicitly defined and validated during installation.

Verification

1. Run the following command to verify that the **DataScienceCluster** has the updated fields:

```
$ oc get datasciencecluster default-dsc -n redhat-ods-operator -o "jsonpath={.data}"
```

2. Run the following command to verify that the **trustyai-dsc-config** ConfigMap has the same flag values set in the **DataScienceCluster**.

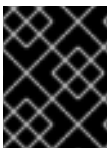
```
$ oc get configmaps trustyai-dsc-config -n redhat-ods-applications -o "jsonpath=
{.spec.components.trustyai.eval.lmeval}"
```

Example output

```
{"eval.lmeval.permitCodeExecution":"true","eval.lmeval.permitOnline":"true"}
```

2.2.2. Updating LMEval job configuration using the web console

Follow these steps to enable online access (**allowOnline**) and remote code execution (**allowCodeExecution**) modes through the OpenShift AI web console for LMEval jobs.



IMPORTANT

Enabling online access or code execution involves a security risk. Only use these configurations if you trust the source(s).

Prerequisites

- You have cluster administrator privileges for your Red Hat OpenShift AI cluster.

Procedure

1. In the OpenShift console, click **Ecosystem → Installed Operators**.
2. Search for the **Red Hat OpenShift AI** Operator, and then click the Operator name to open the Operator details page.
3. Click the **Data Science Cluster** tab.
4. Click the default instance name (for example, **default-dsc**) to open the instance details page.
5. Click the **YAML** tab to show the instance specifications.
6. In the **spec:components:trustyai:eval:lmeval** section, set the **permitCodeExecution** and **permitOnline** fields to a value of **allow**:

```
spec:
  components:
    trustyai:
      managementState: Managed
      eval:
        lmeval:
          permitOnline: allow
          permitCodeExecution: allow
```

7. Click **Save**.
8. From the **Project** drop-down list, select the project that contains the LMEval job you are working with.
9. From the **Resources** drop-down list, select the **LMEvalJob** instance that you are working with.
10. Click **Actions → Edit YAML**
11. Ensure that the **allowOnline** and **allowCodeExecution** are set to **true** to enable online access and code execution for this job when writing your **LMEvalJob** custom resource:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
```

```
kind: LMEvalJob
metadata:
  name: example-lmeval
spec:
  allowOnline: true
  allowCodeExecution: true
```

12. Click **Save**.

Table 2.2. Configuration keys for LMEvalJob custom resource

Field	Default	Description
spec.allowOnline	**false**	Enables this job to access the internet (e.g., to download datasets or tokenizers).
spec.allowCodeExecution	**false**	Allows this job to run code included with downloaded resources.

2.3. LM-EVAL EVALUATION JOB

LM-Eval service defines a new Custom Resource Definition (CRD) called **LMEvalJob**. An **LMEvalJob** object represents an evaluation job. **LMEvalJob** objects are monitored by the TrustyAI Kubernetes operator.

To run an evaluation job, create an **LMEvalJob** object with the following information: **model**, **model arguments**, **task**, and **secret**.

NOTE

For a list of TrustyAI-supported tasks, see LMEval task support.

After the **LMEvalJob** is created, the LM-Eval service runs the evaluation job. The status and results of the **LMEvalJob** object update when the information is available.

NOTE

Other TrustyAI features (such as bias and drift metrics) cannot be used with non-tabular models (including LLMs). Deploying the **TrustyAIService** custom resource (CR) in a namespace that contains non-tabular models (such as the namespace where an evaluation job is being executed) can cause errors within the TrustyAI service.

Sample LMEvalJob object

The sample **LMEvalJob** object contains the following features:

- The **google/flan-t5-base** model from Hugging Face.
- The dataset from the **wnli** card, a subset of the GLUE (General Language Understanding Evaluation) benchmark evaluation framework from Hugging Face. For more information about the **wnli** Unitxt card, see the Unitxt website.

- The following default parameters for the **multi_class.relation** Unitxt task: **f1_micro**, **f1_macro**, and **accuracy**. This template can be found on the Unitxt website: click **Catalog**, then click **Tasks** and select **Classification** from the menu.

The following is an example of an **LMEvalJob** object:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  model: hf
  modelArgs:
  - name: pretrained
    value: google/flan-t5-base
  taskList:
    taskRecipes:
    - card:
        name: "cards.wnli"
      template: "templates.classification.multi_class.relation.default"
  logSamples: true
```

After you apply the sample **LMEvalJob**, check its state by using the following command:

```
oc get lmevaljob evaljob-sample
```

Output similar to the following appears: NAME: **evaljob-sample** STATE: **Running**

Evaluation results are available when the state of the object changes to **Complete**. Both the model and dataset in this example are small. The evaluation job should finish within 10 minutes on a CPU-only node.

Use the following command to get the results:

```
oc get lmevaljobs.trustyai.opendatahub.io evaljob-sample \
  -o template --template={{.status.results}} | jq '.results'
```

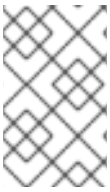
The command returns results similar to the following example:

```
{
  "tr_0": {
    "alias": "tr_0",
    "f1_micro,none": 0.5633802816901409,
    "f1_micro_stderr,none": "N/A",
    "accuracy,none": 0.5633802816901409,
    "accuracy_stderr,none": "N/A",
    "f1_macro,none": 0.36036036036036034,
    "f1_macro_stderr,none": "N/A"
  }
}
```

Notes on the results

- The **f1_micro**, **f1_macro**, and **accuracy** scores are 0.56, 0.36, and 0.56.

- The full results are stored in the **.status.results** of the **LMEvalJob** object as a JSON document.
- The command above only retrieves the results field of the JSON document.



NOTE

The provided **LMEvalJob** uses a dataset from the **wnli** card, which is in Parquet format and not supported on **s390x**. To run on **s390x**, choose a task that uses a non-Parquet dataset.

2.4. LM-EVAL EVALUATION JOB PROPERTIES

The **LMEvalJob** object contains the following features:

- The **google/flan-t5-base** model.
- The dataset from the **wnli** card, from the GLUE (General Language Understanding Evaluation) benchmark evaluation framework.
- The **multi_class.relation** Unitxt task default parameters.

The following table lists each property in the **LMEvalJob** and its usage:

Table 2.3. LM-EvalJob properties

Parameter	Description
model	Specifies which model type or provider is evaluated. This field directly maps to the **--model** argument of the**lm-evaluation-harness**. The model types and providers that you can use include: • **hf**: HuggingFace models • **openai-completions**: OpenAI Completions API models • **openai-chat-completions**: OpenAI Chat Completions API models • **local-completions** and **local-chat-completions**: OpenAI API-compatible servers • **textsynth**: TextSynth APIs

Parameter	Description
modelArgs	A list of paired name and value arguments for the model type. Arguments vary by model provider. You can find further details in the models section of the LM Evaluation Harness library on GitHub. Below are examples for some providers: • **hf**: The model designation for the HuggingFace provider • **local-completions**: An OpenAI API-compatible server • **local-chat-completions**: An OpenAI API-compatible server • **openai-completions**: OpenAI Completions API models • **openai-chat-completions**: ChatCompletions API models • **textsynth**: TextSynth APIs
taskList.taskNames	Specifies a list of tasks supported by **lm-evaluation-harness**.

Parameter	Description

taskList.taskRecipes	Specifies the task using the Unitxt recipe format:
	• **card**: Use the **name** to specify a Unitxt card or **ref** to refer to a custom card:
	  ◦ **name**: Specifies a Unitxt card from the catalog section of the Unitxt. Use the card ID as the value. For example, the ID of the Wnli card is **cards.wnli**.
	  ◦ **ref**: Specifies the reference name of a custom card as defined in the **custom** section. If the dataset used by the custom card requires an API key from an environment variable or a persistent volume, configure the necessary resources in the **pod** field.
	• **template**: Specifies a Unitxt template from the Unitxt catalog. Use **name** to specify a Unitxt catalog template or **ref** to refer to a custom template:
	  ◦ **name**: Specifies a Unitxt template from the catalog of cards on the Unitxt website. Use the template's ID as the value.
	  ◦ **ref**: Specifies the reference name of a custom template as defined in the **custom** section.
	• **systemPrompt**: Use **name** to specify a Unitxt catalog system prompt or **ref** to refer to a custom prompt:
	  ◦ **name**: Specifies a Unitxt system prompt from the catalog on the Unitxt website. Use the system prompt's ID as the value.
	  ◦ **ref**: Specifies the reference name of a custom system prompt as defined in the **custom** section.
	• **task** (optional): Specifies a Unitxt task from the Unitxt catalog. Use the task ID as the value. A Unitxt card has a predefined task. Only specify a value for this if you want to run a different task.
	• **metrics** (optional): Specifies a Unitxt task from the Unitxt catalog. Use the metric ID as the value. A Unitxt task has a set of pre-defined metrics. Only specify a set of metrics if you need different metrics.
	• **format** (optional): Specifies a Unitxt format from the Unitxt catalog. Use the format ID as the value.
	• **loaderLimit** (optional): Specifies the maximum number of instances per stream to be returned from the loader. You can use this parameter to reduce loading time in large datasets.
	• **numDemos** (optional): Number of few-shot to be used.
	• **demosPoolSize** (optional): Size of the few-shot pool.

numFewShot	Sets the number of few-shot examples to place in context. If you are using a task from Unitxt, do not use this field. Use **numDemos** under the **taskRecipes** instead.

Parameter	Description
limit	Set a limit to run the tasks instead of running the entire dataset. Accepts either an integer or a float between **0.0** and **1.0**.
genArgs	Maps to the **--gen_kwargs** parameter for the **lm-evaluation-harness**. For more information, see the LM Evaluation Harness documentation on GitHub.
logSamples	If this flag is passed, then the model outputs and the text fed into the model are saved at per-prompt level.
batchSize	Specifies the batch size for the evaluation in integer format. The **auto:N** batch size is not used for API models, but numeric batch sizes are used for APIs.

pod	Specifies extra information for the **lm-eval** job pod:
	• **container**: Specifies additional container settings for the **lm-eval** container.
	  ◦ **env**: Specifies environment variables. This parameter uses the **EnvVar** data structure of Kubernetes.
	  ◦ **volumeMounts**: Mounts the volumes into the **lm-eval** container.
	  ◦ **resources**: Specifies the resources for the **lm-eval** container.
	• **volumes**: Specifies the volume information for the **lm-eval** and other containers. This parameter uses the **Volume** data structure of Kubernetes.
	• **sideCars**: A list of containers that run along with the **lm-eval** container. This parameter uses the **Container** data structure of Kubernetes.

outputs	This parameter defines a custom output location to store the the evaluation results. Only Persistent Volume Claims (PVC) are supported.
outputs.pvcManaged	Creates an operator-managed PVC to store the job results. The PVC is named **<job-name>-pvc** and is owned by the **LMEvalJob**. After the job finishes, the PVC is still available, but it is deleted with the **LMEvalJob**. Supports the following fields: • **size**: The PVC size, compatible with standard PVC syntax (for example, 5Gi).
outputs.pvcName	Binds an existing PVC to a job by specifying its name. The PVC must be created separately and must already exist when creating the job.

Parameter	Description
allowOnline	If this parameter is set to **true**, the LMEval job downloads artifacts as needed (for example, models, datasets or tokenizers). If set to **false**, artifacts are not downloaded and are pulled from local storage instead. This setting is disabled by default. If you want to enable **allowOnline** mode, you can deploy a new **LMEvalJob** CR with **allowOnline** set to **true** as long as the **DataScienceCluster** resource specification **permitOnline** is also set to **true**.
allowCodeExecution	If this parameter is set to **true**, the LMEval job runs the necessary code for preparing models or datasets. If set to **false** it does not run downloaded code. The default setting for this parameter is **false**. If you want to enable **allowCodeExecution** mode, you can deploy a new**LMEvalJob** CR with **allowCodeExecution** set to **true** as long as the **DataScienceCluster** resource specification **permitCodeExecution** is also set to **true**.
offline	Mount a PVC as the local storage for models and datasets.
systemInstruction	(Optional) Sets the system instruction for all prompts passed to the evaluated model.
chatTemplate	Applies the specified chat template to prompts. Contains two fields: * **enabled**: If set to **true**, a chat template is used. If set to**false**, no template is used. * **name**: Uses the template name, if provided. If no name argument is provided, uses the default template for the model.

2.4.1. Properties for setting up custom Unitxt cards, templates, or system prompts

You can choose to set up custom Unitxt cards, templates, or system prompts. Use the parameters set out in the Custom Unitxt parameters table in addition to the preceding table parameters to set customized Unitxt items:

Table 2.4. Custom Unitxt parameters

Parameter	Description

Parameter	Description

taskList.custom	Defines one or more custom resources that is referenced in a task recipe. The following custom cards, templates, and system prompts are supported:
	• **cards**: Defines custom cards to use, each with a **name** and **value** field:
	  ◦ **name**: The name of this custom card that is referenced in the **card.ref** field of a task recipe.
	  ◦ **value**: A JSON string for a custom Unitxt card that contains the custom dataset. To compose a custom card, store it as a JSON file, and use the JSON content as the value. If the dataset used by the custom card needs an API key from an environment variable or a persistent volume, set up corresponding resources under the **pod** field in the **LMEvalJob`** properties table.
	• **templates**: Define custom templates to use, each with a **name** and **value** field:
	  ◦ **name**: The name of this custom template that is referenced in the **template.ref** field of a task recipe.
	  ◦ **value**: A JSON string for a custom Unitxt template. Store **value** as a JSON file and use the JSON content as the value of this field.
	• **systemPrompts**: Defines custom system prompts to use, each with a **name** and **value** field:
	  ◦ **name**: The name of this custom system prompt that is referenced in the **systemPrompt.ref** field of a task recipe.
	  ◦ **value**: A string for a custom Unitxt system prompt. You can see an overview of the different components that make up a prompt format, including the system prompt, on the Unitxt website.

2.5. PERFORMING MODEL EVALUATIONS IN THE DASHBOARD

LM-Eval is a Language Model Evaluation as a Service (LM-Eval-aaS) feature integrated into the TrustyAI Operator. It offers a unified framework for testing generative language models across a wide variety of evaluation tasks. You can use LM-Eval through the Red Hat OpenShift AI dashboard or the OpenShift CLI (**oc**). These instructions are for using the dashboard.

IMPORTANT

Model evaluation through the dashboard is currently available in Red Hat OpenShift AI 3.4 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process. For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

Prerequisites

- You have logged in to Red Hat OpenShift AI with administrator privileges.
- You have enabled the TrustyAI component, as described in Enabling the TrustyAI component.
- You have created a project in OpenShift AI.
- You have deployed an LLM model in your project.

NOTE

By default, the **Develop & train → Evaluations** page is hidden from the dashboard navigation menu. To show the **Develop & train → Evaluations** page in the dashboard, go to the **OdhDashboardConfig** custom resource (CR) in Red Hat OpenShift AI and set the **disableLMEval** value to false. For more information about enabling dashboard configuration options, see Dashboard configuration options .

Procedure

1. In the dashboard, click **Develop & train → Evaluations**. The Evaluations page opens. It contains:
   a. A **Start evaluation run** button. If you have not run any previous evaluations, only this button is displayed.
   b. A list of evaluations you have previously run, if any exist.
   c. A **Project** dropdown option you can click to show the evaluations relating to one project instead of all projects.
   d. A filter to sort your evaluations by model or evaluation name.

   The following table outlines the elements and functions of the evaluations list:

Table 2.5. Evaluations list components

Property	Function
Evaluation	The name of the evaluation.
Model	The model that was used in the evaluation.
Evaluated	The date and time when the evaluation was created.

Property	Function
Status	The status of your evaluation: running, completed, or failed.
More options icon	Click this icon to access the options to delete the evaluation, or download the evaluation log in JSON format.

2. From the **Project** dropdown menu, select the namespace of the project where you want to evaluate the model.
3. Click the **Start evaluation run** button. The Model evaluation form is displayed.
4. Fill in the details of the form. The model argument summary is displayed after you complete the form details:
    a. **Model name**: Select a model from all the deployed LLMs in your project.
    b. **Evaluation name**: Give your evaluation a unique name.
    c. **Tasks**: Choose one or more evaluation tasks against which to measure your LLM. The 100 most common evaluation tasks are supported.
    d. **Model type**: Choose the type of model based on the type of prompt-formatting you use:
        i. **Local-completion**: You assemble the entire prompt chain yourself. Use this when you want to evaluate models that take a plain text prompt and return a continuation.
        ii. **Local-chat-completion**: The framework injects roles or templates automatically. Use this for models that simulate a conversation by taking a list of chat messages with roles like **user** and **assistant** and reply appropriately.
    e. **Security settings**:
        i. **Available online**: Choose **enable** to allow your model to access the internet to download datasets.
        ii. **Trust remote code**: Choose **enable** to allow your model to trust code from outside of the project namespace.

NOTE

The **Security settings** section is grayed out if the security option in global settings is set to **active**.

5. Observe that a model argument summary is displayed as soon as you fill in the form details.
6. Complete the tokenizer settings:
    a. **Tokenized requests**: If set to **true**, the evaluation requests are broken down into tokens. If set to **false**, the evaluation dataset remains as raw text.
    b. **Tokenizer**: Type the model's tokenizer URL that is required for the evaluations.

7. Click **Evaluate**. The screen returns to the model evaluation page of your project and your job is displayed in the evaluations list.

NOTE

- It can take time for your evaluation to complete, depending on factors including hardware support, model size, and the type of evaluation task(s). The status column reports the current status of the evaluation: *completed*, *running*, or *failed*.
- If your evaluation fails, the evaluation pod logs in your cluster provide more information.

2.6. LM-EVAL METRICS

Use LM-Eval metrics to track functions and outputs of your LM-Eval deployment and understand how your model is working. Metrics are included as standard in your LM-Eval deployment.

Table 2.6. LM-Eval metrics

Metric	Labels	Description
trustyai_eval	• **eval_job_namespace**: namespace into which the evaluation job was deployed • **framework**: the evaluation framework used by the job, for example **lm-evaluation-harness** • **model_type**: the model type being evaluated, for example **local-chat-completions** • **task**: the evaluation task being performed, for example **mmlu**	Tracks the total number of LM-Eval jobs that have been deployed into the cluster, grouped by attributes of the job.

2.7. LM-EVAL SCENARIOS

The following procedures outline example scenarios that can be useful for an LM-Eval setup.

2.7.1. Accessing Hugging Face models with an environment variable token

If the **LMEvalJob** needs to access a model on HuggingFace with the access token, you can set up the **HF_TOKEN** as one of the environment variables for the **lm-eval** container.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.

Procedure

1. To start an evaluation job for a **huggingface** model, apply the following YAML file to your project through the CLI:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  model: hf
  modelArgs:
  - name: pretrained
    value: huggingfacespace/model
  taskList:
    taskNames:
    - unfair_tos/
  logSamples: true
  pod:
    container:
      env:
      - name: HF_TOKEN
        value: "My HuggingFace token"
```

   For example:

```
$ oc apply -f <yaml_file> -n <project_name>
```

2. (Optional) You can also create a secret to store the token, then refer the key from the **secretKeyRef** object using the following reference syntax:

```
env:
  - name: HF_TOKEN
    valueFrom:
      secretKeyRef:
        name: my-secret
        key: hf-token
```

2.7.2. Using a custom Unitxt card

You can run evaluations using custom Unitxt cards. To do this, include the custom Unitxt card in JSON format within the **LMEvalJob** YAML.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.

Procedure

1. Pass a custom Unitxt Card in JSON format:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  model: hf
  modelArgs:
  - name: pretrained
    value: google/flan-t5-base
  taskList:
    taskRecipes:
    - template: "templates.classification.multi_class.relation.default"
      card:
        custom: |
          {
            "__type__": "task_card",
            "loader": {
              "__type__": "load_hf",
              "path": "glue",
              "name": "wnli"
            },
            "preprocess_steps": [
              {
                "__type__": "split_random_mix",
                "mix": {
                  "train": "train[95%]",
                  "validation": "train[5%]",
                  "test": "validation"
                }
              },
              {
                "__type__": "rename",
                "field": "sentence1",
                "to_field": "text_a"
              },
              {
                "__type__": "rename",
                "field": "sentence2",
                "to_field": "text_b"
              },
              {
                "__type__": "map_instance_values",
                "mappers": {
                  "label": {
                    "0": "entailment",
                    "1": "not entailment"
                  }
                }
              },
              {
                "__type__": "set",
                "fields": {
                  "classes": [
                    "entailment",
                    "not entailment"
                  ]
```

```
            }
          },
          {
            "__type__": "set",
            "fields": {
              "type_of_relation": "entailment"
            }
          },
          {
            "__type__": "set",
            "fields": {
              "text_a_type": "premise"
            }
          },
          {
            "__type__": "set",
            "fields": {
              "text_b_type": "hypothesis"
            }
          }
        ],
        "task": "tasks.classification.multi_class.relation",
        "templates": "templates.classification.multi_class.relation.all"
      }
logSamples: true
```

2. Inside the custom card specify the Hugging Face dataset loader:

```
"loader": {
            "__type__": "load_hf",
            "path": "glue",
            "name": "wnli"
          },
```

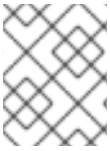
3. (Optional) You can use other Unitxt loaders (found on the Unitxt website) that contain the **volumes** and **volumeMounts** parameters to mount the dataset from persistent volumes. For example, if you use the **LoadCSV** Unitxt command, mount the files to the container and make the dataset accessible for the evaluation process.

NOTE

The provided scenario example does not work on **s390x**, as it uses a Parquet-type dataset, which is not supported on this architecture. To run the scenario on **s390x**, use a task with a non-Parquet dataset.

2.7.3. Using PVCs as storage

To use a PVC as storage for the **LMEvalJob** results, you can use either managed PVCs or existing PVCs. Managed PVCs are managed by the TrustyAI operator. Existing PVCs are created by the end-user before the **LMEvalJob** is created.



NOTE

If both managed and existing PVCs are referenced in outputs, the TrustyAI operator defaults to the managed PVC.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.

2.7.3.1. Managed PVCs

To create a managed PVC, specify its size. The managed PVC is named **<job-name>-pvc** and is available after the job finishes. When the **LMEvalJob** is deleted, the managed PVC is also deleted.

Procedure

- Enter the following code:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  # other fields omitted ...
  outputs:
    pvcManaged:
      size: 5Gi
```

Notes on the code

- **outputs** is the section for specifying custom storage locations
- **pvcManaged** will create an operator-managed PVC
- **size** (compatible with standard PVC syntax) is the only supported value

2.7.3.2. Existing PVCs

To use an existing PVC, pass its name as a reference. The PVC must exist when you create the **LMEvalJob**. The PVC is not managed by the TrustyAI operator, so it is available after deleting the **LMEvalJob**.

Procedure

1. Create a PVC. An example is the following:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: "my-pvc"
spec:
```

```
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

2. Reference the new PVC from the **LMEvalJob**.

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  # other fields omitted ...
  outputs:
    pvcName: "my-pvc"
```

2.7.4. Using a KServe Inference Service

To run an evaluation job on an **InferenceService** which is already deployed and running in your namespace, define your **LMEvalJob** CR, then apply this CR into the same namespace as your model.

NOTE

The following example only works with Hugging Face or vLLM-based model-serving runtimes.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.
- You have a namespace that contains an InferenceService with a vLLM model. This example assumes that a vLLM model is already deployed in your cluster.
- Your cluster has Domain Name System (DNS) configured.

Procedure

1. Define your **LMEvalJob** CR:

```
  apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob
spec:
  model: local-completions
  taskList:
    taskNames:
      - mmlu
  logSamples: true
  batchSize: 1
  modelArgs:
    - name: model
```

```
      value: granite
    - name: base_url
      value: $ROUTE_TO_MODEL/v1/completions
    - name: num_concurrent
      value:  "1"
    - name: max_retries
      value:  "3"
    - name: tokenized_requests
      value: false
    - name: tokenizer
      value: huggingfacespace/model
env:
  - name: OPENAI_TOKEN
    valueFrom:
        secretKeyRef:
          name: <secret-name>
          key: token
```

2. Apply this CR into the same namespace as your model.

Verification

A pod spins up in your model namespace called **evaljob**. In the pod terminal, you can see the output via **tail -f output/stderr.log**.

Notes on the code

- **base_url** should be set to the route/service URL of your model. Make sure to include the /**v1**/**completions** endpoint in the URL.
- **env.valueFrom.secretKeyRef.name** should point to a secret that contains a token that can authenticate to your model. **secretRef.name** should be the secret's name in the namespace, while **secretRef.key** should point at the token's key within the secret.
- **secretKeyRef.name** can equal the output of:

```
oc get secrets -o custom-columns=SECRET:.metadata.name --no-headers | grep user-one-token
```

- **secretKeyRef.key** is set to **token**

2.7.5. Setting up LM-Eval S3 Support

Learn how to set up S3 support for your LM-Eval service.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.
- You have a namespace that contains an S3-compatible storage service and bucket.

- You have created an **LMEvalJob** that references the S3 bucket containing your model and dataset.
- You have an S3 bucket that contains the model files and the dataset(s) to be evaluated.

Procedure

1. Create a Kubernetes Secret containing your S3 connection details:

```
apiVersion: v1
kind: Secret
metadata:
  name: "s3-secret"
  namespace: test
  labels:
    opendatahub.io/dashboard: "true"
    opendatahub.io/managed: "true"
  annotations:
    opendatahub.io/connection-type: s3
    openshift.io/display-name: "S3 Data Connection - LMEval"
data:
  AWS_ACCESS_KEY_ID: BASE64_ENCODED_ACCESS_KEY  # Replace with your key
  AWS_SECRET_ACCESS_KEY: BASE64_ENCODED_SECRET_KEY  # Replace with
your key
  AWS_S3_BUCKET: BASE64_ENCODED_BUCKET_NAME  # Replace with your bucket
name
  AWS_S3_ENDPOINT: BASE64_ENCODED_ENDPOINT  # Replace with your endpoint
URL (for example,  https://s3.amazonaws.com)
  AWS_DEFAULT_REGION: BASE64_ENCODED_REGION  # Replace with your region
type: Opaque
```

NOTE

All values must be **base64** encoded. For example: **echo -n "my-bucket" | base64**

2. Deploy the **LMEvalJob** CR that references the S3 bucket containing your model and dataset:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
  name: evaljob-sample
spec:
  allowOnline: false
  model: hf  # Model type (HuggingFace in this example)
  modelArgs:
    - name: pretrained
      value: /opt/app-root/src/hf_home/flan  # Path where model is mounted in container
  taskList:
    taskNames:
      - arc_easy  # The evaluation task to run
  logSamples: true
  offline:
    storage:
      s3:
```

```
      accessKeyId:
        name: s3-secret
        key: AWS_ACCESS_KEY_ID
      secretAccessKey:
        name: s3-secret
        key: AWS_SECRET_ACCESS_KEY
      bucket:
        name: s3-secret
        key: AWS_S3_BUCKET
      endpoint:
        name: s3-secret
        key: AWS_S3_ENDPOINT
      region:
        name: s3-secret
        key: AWS_DEFAULT_REGION
      path: ""  # Optional subfolder within bucket
      verifySSL: false
```

IMPORTANT

The `LMEvalJob` will copy all the files from the specified bucket/path. If your bucket contains many files and you only want to use a subset, set the `path` field to the specific sub-folder containing the files that you require. For example use `path: "my-models/"`.

3. Set up a secure connection using SSL.

   a. Create a ConfigMap object with your CA certificate:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: s3-ca-cert
  namespace: test
  annotations:
    service.beta.openshift.io/inject-cabundle: "true"  # For injection
data: {}  # OpenShift will inject the service CA bundle
# Or add your custom CA:
# data:
#   ca.crt: |-
#     -----BEGIN CERTIFICATE-----
#     ...your CA certificate content...
#     -----END CERTIFICATE-----
```

   b. Update the **LMEvalJob** to use SSL verification:

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
    name: evaljob-sample
spec:
    # ... same as above ...
    offline:
        storage:
```

```
s3:
  # ... same as above ...
  verifySSL: true  # Enable SSL verification
  caBundle:
    name: s3-ca-cert  # ConfigMap name containing your CA
    key: service-ca.crt  # Key in ConfigMap containing the certificate
```

Verification

1. After deploying the **LMEvalJob**, open the **kubectl** command-line and enter this command to check its status: **kubectl logs -n test job/evaljob-sample -n test**
2. View the logs with the **kubectl** command **kubectl logs -n test job/<job-name>** to make sure it has functioned correctly.
3. The results are displayed in the logs after the evaluation is completed.

2.7.6. Using LLM-as-a-Judge metrics with LM-Eval

You can use a large language model (LLM) to assess the quality of outputs from another LLM, known as LLM-as-a-Judge (LLMaaJ).

You can use LLMaaJ to:

- Assess work with no clearly correct answer, such as creative writing.
- Judge quality characteristics such as helpfulness, safety, and depth.
- Augment traditional quantitative measures that are used to evaluate a model's performance (for example, **ROUGE** metrics).
- Test specific quality aspects of your model output.

Follow the custom quality assessment example below to learn more about using your own metrics criteria with LM-Eval to evaluate model responses.

This example uses Unitxt to define custom metrics and to see how the model (flan-t5-small) answers questions from MT-Bench, a standard benchmark. Custom evaluation criteria and instructions from the Mistral-7B model are used to rate the answers from 1-10, based on helpfulness, accuracy, and detail.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- Your cluster administrator has installed OpenShift AI and enabled the TrustyAI service for the project where the models are deployed.
- You are familiar with how to use Unitxt.

- You have set the following parameters:

  Table 2.7. Parameters

Parameter	Description
Custom template	Tells the judge to assign a score between 1 and 10 in a standardized format, based on specific criteria.
processors.extract_mt_bench_rating_judgment	Pulls the numerical rating from the judge's response.
formats.models.mistral.instruction	Formats the prompts for the Mistral model.
Custom LLM-as-judge metric	Uses Mistral-7B with your custom instructions.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

   ```
   $ oc login <openshift_cluster_url> -u <admin_username> -p <password>
   ```

2. Apply the following manifest by using the **oc apply -f -** command. The YAML content defines a custom evaluation job (**LMEvalJob**), the namespace, and the location of the model you want to evaluate. The YAML contains the following instructions:

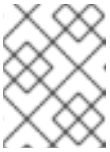
   a. Which model to evaluate.

   b. What data to use.

   c. How to format inputs and outputs.

   d. Which judge model to use.

   e. How to extract and log results.

   NOTE

   You can also put the YAML manifest into a file using a text editor and then apply it by using the **oc apply -f file.yaml** command.

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: LMEvalJob
metadata:
 name: custom-eval
 namespace: test
spec:
 allowOnline: true
 allowCodeExecution: true
 model: hf
```

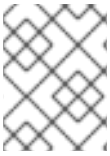
```
 modelArgs:
   - name: pretrained
     value: google/flan-t5-small
taskList:
 taskRecipes:
     - card:
         custom: |
           {
              "__type__": "task_card",
              "loader": {
                 "__type__": "load_hf",
                 "path": "OfirArviv/mt_bench_single_score_gpt4_judgement",
                 "split": "train"
              },
              "preprocess_steps": [
                 {
                    "__type__": "rename_splits",
                    "mapper": {
                       "train": "test"
                    }
                 },
                 {
                    "__type__": "filter_by_condition",
                    "values": {
                       "turn": 1
                    },
                    "condition": "eq"
                 },
                 {
                    "__type__": "filter_by_condition",
                    "values": {
                       "reference": "[]"
                    },
                    "condition": "eq"
                 },
                 {
                    "__type__": "rename",
                    "field_to_field": {
                       "model_input": "question",
                       "score": "rating",
                       "category": "group",
                       "model_output": "answer"
                    }
                 },
                 {
                    "__type__": "literal_eval",
                    "field": "question"
                 },
                 {
                    "__type__": "copy",
                    "field": "question/0",
                    "to_field": "question"
                 },
                 {
                    "__type__": "literal_eval",
                    "field": "answer"
```

```
                },
                {
                    "__type__": "copy",
                    "field": "answer/0",
                    "to_field": "answer"
                }
            ],
            "task": "tasks.response_assessment.rating.single_turn",
            "templates": [
                "templates.response_assessment.rating.mt_bench_single_turn"
            ]
        }
      template:
        ref: response_assessment.rating.mt_bench_single_turn
      format: formats.models.mistral.instruction
      metrics:
      - ref: llmaaj_metric
  custom:
    templates:
      - name: response_assessment.rating.mt_bench_single_turn
        value: |
          {
            "__type__": "input_output_template",
            "instruction": "Please act as an impartial judge and evaluate the quality of the response
provided by an AI assistant to the user question displayed below. Your evaluation should consider
factors such as the helpfulness, relevance, accuracy, depth, creativity, and level of detail of the
response. Begin your evaluation by providing a short explanation. Be as objective as possible. After
providing your explanation, you must rate the response on a scale of 1 to 10 by strictly following this
format: \"[[rating]]\", for example: \"Rating: [[5]]\".\n\n",
            "input_format": "[Question]\n{question}\n\n[The Start of Assistant's Answer]\n{answer}\n[The
End of Assistant's Answer]",
            "output_format": "[[{rating}]]",
            "postprocessors": [
                "processors.extract_mt_bench_rating_judgment"
            ]
        }
    tasks:
      - name: response_assessment.rating.single_turn
        value: |
          {
            "__type__": "task",
            "input_fields": {
                "question": "str",
                "answer": "str"
            },
            "outputs": {
                "rating": "float"
            },
            "metrics": [
                "metrics.spearman"
            ]
        }
    metrics:
      - name: llmaaj_metric
        value: |
          {
```

```
            "__type__": "llm_as_judge",
            "inference_model": {
                "__type__": "hf_pipeline_based_inference_engine",
                "model_name": "mistralai/Mistral-7B-Instruct-v0.2",
                "max_new_tokens": 256,
                "use_fp16": true
            },
            "template": "templates.response_assessment.rating.mt_bench_single_turn",
            "task": "rating.single_turn",
            "format": "formats.models.mistral.instruction",
            "main_score": "mistral_7b_instruct_v0_2_huggingface_template_mt_bench_single_turn"
        }
logSamples: true
pod:
  container:
    env:
      - name: HF_TOKEN
        valueFrom:
          secretKeyRef:
            name: hf-token-secret
            key: token
    resources:
      limits:
        cpu: '2'
        memory: 16Gi
```

Verification

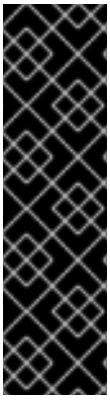
A processor extracts the numeric rating from the judge's natural language response. The final result is available as part of the LMEval Job Custom Resource (CR).



NOTE

The provided scenario example does not work for **s390x**. The scenario works with non-Parquet type dataset task for **s390x**.

CHAPTER 3. EVALUATING RAG SYSTEMS WITH RAGAS



IMPORTANT

Retrieval-Augmented Generation Assessment (Ragas) is currently available in Red Hat OpenShift AI as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

As an AI engineer, you can use Retrieval-Augmented Generation Assessment (Ragas) to measure and improve the quality of your RAG systems in OpenShift AI. Ragas provides objective metrics that assess retrieval quality, answer relevance, and factual consistency, enabling you to identify issues, optimize configurations, and establish automated quality gates in your development workflows.

Ragas is integrated with OpenShift AI through the Llama Stack evaluation API and supports two deployment modes: an inline provider for development and testing, and a remote provider for production-scale evaluations using OpenShift AI pipelines.

3.1. ABOUT RAGAS EVALUATION

Ragas addresses the unique challenges of evaluating RAG systems by providing metrics that assess both the retrieval and generation components of your application. Unlike traditional language model evaluation that focuses solely on output quality, Ragas evaluates how well your system retrieves relevant context and generates responses grounded in that context.

3.1.1. Key Ragas metrics

Ragas provides multiple metrics for evaluating RAG systems. Here are some of the metrics:

Faithfulness

Measures the generated answer to determine whether it is consistent with the retrieved context. A high faithfulness score indicates that the answer is well-grounded in the source documents, reducing the risk of hallucinations. This is critical for enterprise and regulated environments where accuracy and trustworthiness are paramount.

Answer Relevancy

Evaluates whether the generated answer is consistent with the input question. This metric ensures that your RAG system provides pertinent responses rather than generic or off-topic information.

Context Precision

Measures the precision of the retrieval component by evaluating whether the retrieved context chunks contain information relevant to answering the question. High precision indicates that your retrieval system is returning focused, relevant documents rather than irrelevant noise.

Context Recall

Measures the recall of the retrieval component by evaluating whether all necessary information for answering the question is present in the retrieved contexts. High recall ensures that your retrieval system is not missing important information.

Answer Correctness

Compares the generated answer with a ground truth reference answer to measure accuracy. This metric is useful when you have labeled evaluation datasets with known correct answers.

Answer Similarity

Measures the semantic similarity between the generated answer and a reference answer, providing a more nuanced assessment than exact string matching.

3.1.2. Use cases for Ragas in AI engineering workflows

Ragas enables AI engineers to accomplish the following tasks:

Automate quality checks

Create reproducible, objective evaluation jobs that can be automatically triggered after every code commit or model update. Automatic quality checks establish quality gates to prevent regressions and ensure that you deploy only high-quality RAG configurations to production.

Enable evaluation-driven development (EDD)

Use Ragas metrics to guide iterative optimization. For example, test different chunking strategies, embedding models, or retrieval algorithms against a defined benchmark. You can discover the optimal RAG configuration that maximizes performance metrics. For example, you can maximize faithfulness while minimizing computational cost.

Ensure factual consistency and trustworthiness

Measure the reliability of your RAG system by setting thresholds on metrics like faithfulness. Metrics thresholds ensure that responses are consistently grounded in source documents, which is critical for enterprise applications where hallucinations or factual errors are unacceptable.

Achieve production scalability

Leverage the remote provider pattern with OpenShift AI pipelines to execute evaluations as distributed jobs. The remote provider pattern allows you to run large-scale benchmarks across thousands of data points without blocking development or consuming excessive local resources.

Compare model and configuration variants

Run comparative evaluations across different models, retrieval strategies, or system configurations to make data-driven decisions about your RAG architecture. For example, compare the impact of different chunk sizes (512 vs 1024 tokens) or different embedding models on retrieval quality metrics.

3.1.3. Ragas provider deployment modes

OpenShift AI supports two deployment modes for Ragas evaluation:

Inline provider

The inline provider mode runs Ragas evaluation in the same process as the Llama Stack server. Use the inline provider for development and rapid prototyping. It offers the following advantages:

- Fast processing with in-memory operations
- Minimal configuration overhead
- Local development and testing
- Evaluation of small to medium-sized datasets

Remote provider

The remote provider mode runs Ragas evaluation as distributed jobs using OpenShift AI pipelines (powered by Kubeflow Pipelines). Use the remote provider for production environments. It offers the following capabilities:

- Running evaluations in parallel across thousands of data points
- Providing resource isolation and management
- Integrating with CI/CD pipelines for automated quality gates
- Storing results in S3-compatible object storage
- Tracking evaluation history and metrics over time
- Supporting large-scale batch evaluations without impacting the Llama Stack server

3.2. SETTING UP THE RAGAS INLINE PROVIDER FOR DEVELOPMENT

You can set up the Ragas inline provider to run evaluation workloads directly inside the Llama Stack server pod. The inline provider is intended for development and experimentation, where simplicity and rapid iteration are more important than scalability or isolation.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**).
- You have activated the Llama Stack Operator in OpenShift AI. For more information, see Activating the Llama Stack Operator.
- You have deployed a Llama model with KServe. For more information, see Deploying a Llama model with KServe.
- You have created a project.

Procedure

1. Log in to your OpenShift cluster if you are not already logged in:

```
oc login <openshift_cluster_url>
```

2. Switch to your project:

```
oc project <project_name>
```

3. Create a **LlamaStackDistribution** that enables the Ragas inline provider by setting the required environment variables. For example, create a file named **llama-stack-ragas-inline.yaml**:

Example LlamaStackDistribution with Ragas inline provider

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
```

```
    name: llama-stack-ragas-inline
    namespace: <project_name>
  spec:
    replicas: 1
    server:
      containerSpec:
        env:
          - name: INFERENCE_MODEL
            value: <model_name>
          - name: VLLM_URL
            value: <model_url>
          - name: VLLM_API_TOKEN
            value: <model_api_token>
          - name: VLLM_TLS_VERIFY
            value: "false"

          # Ragas inline configuration
          - name: TRUSTYAI_EMBEDDING_MODEL
            value: <embedding_model>
        name: llama-stack
        port: 8321
      distribution:
        name: rh-dev
```

4. Deploy the Llama Stack distribution:

```
oc apply -f llama-stack-ragas-inline.yaml
```

5. Monitor the deployment until the pod is running:

```
oc get pods -w
```

Verification

- The **llama-stack-ragas-inline** pod reaches the **Running** state.
- The pod logs show the Llama Stack server starting successfully with Ragas enabled.

3.3. CONFIGURING THE RAGAS REMOTE PROVIDER FOR PRODUCTION

You can configure the Ragas remote provider to run evaluations as distributed jobs using OpenShift AI pipelines. The remote provider enables production-scale evaluations by running Ragas in a separate Kubeflow Pipelines environment, providing resource isolation, improved scalability, and integration with CI/CD workflows.

The Ragas remote provider operates independently of the underlying vector store and works with any vector store supported by Llama Stack, including Milvus, FAISS, and PostgreSQL with the pgvector extension.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.

- You have installed the OpenShift AI Operator.
- You have a **DataScienceCluster** custom resource in your environment; in the **spec.components** section the **llamastackoperator.managementState** is enabled with a value of **Managed**.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have configured a pipeline server in your project. For more information, see Configuring a pipeline server.
- You have activated the Llama Stack Operator in OpenShift AI. For more information, see Activating the Llama Stack Operator .
- You have deployed a Large Language Model with KServe. For more information, see Deploying a Llama model with KServe.
- You have configured S3-compatible object storage for storing evaluation results and you know your S3 credentials: AWS access key, AWS secret access key, and AWS default region. For more information, see Adding a connection to your project .
- You have created a project.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <username> -p <password>
```

2. Navigate to your project:

```
$ oc project <project_name>
```

3. Create a secret for storing S3 credentials:

```
$ oc create secret generic "<ragas_s3_credentials>" \
  --from-literal=AWS_ACCESS_KEY_ID=<your_access_key> \
  --from-literal=AWS_SECRET_ACCESS_KEY=<your_secret_key> \
  --from-literal=AWS_DEFAULT_REGION=<your_region>
```

IMPORTANT

Replace the placeholder values with your actual S3 credentials. These AWS credentials are required in two locations:

- In the Llama Stack server pod (as environment variables) - to access S3 when creating pipeline runs.
- In the Kubeflow Pipeline pods (via the secret) - to store evaluation results to S3 during pipeline execution.

The LlamaStackDistribution configuration loads these credentials from the **" <ragas_s3_credentials>"** secret and makes them available to both locations.

4. Create a secret for the Kubeflow Pipelines API token:
   a. Get your token by running the following command:

```
$ export KUBEFLOW_PIPELINES_TOKEN=$(oc whoami -t)
```

   b. Create the secret by running the following command:

```
$ oc create secret generic kubeflow-pipelines-token \
  --from-literal=KUBEFLOW_PIPELINES_TOKEN="$KUBEFLOW_PIPELINES_TOKEN"
```

IMPORTANT

The Llama Stack distribution service account does not have privileges to create pipeline runs. This secret provides the necessary authentication token for creating and managing pipeline runs.

5. Verify that the Kubeflow Pipelines endpoint is accessible:

```
$ curl -k -H "Authorization: Bearer $KUBEFLOW_PIPELINES_TOKEN" \
 https://$KUBEFLOW_PIPELINES_ENDPOINT/apis/v1beta1/healthz
```

6. Create a secret for storing your inference model information:

```
$ export INFERENCE_MODEL="llama-3-2-3b"
$ export VLLM_URL="https://llama-32-3b-instruct-predictor:8443/v1"
$ export VLLM_TLS_VERIFY="false"  # Use "true" in production
$ export VLLM_API_TOKEN="<token_identifier>"

$ oc create secret generic llama-stack-inference-model-secret \
  --from-literal INFERENCE_MODEL="$INFERENCE_MODEL" \
  --from-literal VLLM_URL="$VLLM_URL" \
  --from-literal VLLM_TLS_VERIFY="$VLLM_TLS_VERIFY" \
  --from-literal VLLM_API_TOKEN="$VLLM_API_TOKEN"
```

7. Get the Kubeflow Pipelines endpoint by running the following command and searching for "pipeline" in the routes. This is used in a later step for creating a ConfigMap for the Ragas remote provider configuration:

```
$ oc get routes -A | grep -i pipeline
```

This output should show that the namespace, which is the namespace you specified for **KUBEFLOW_NAMESPACE**, has the pipeline server endpoint and the associated metadata one. The one to use is **ds-pipeline-dspa**.

8. Create a ConfigMap for the Ragas remote provider configuration. For example, create a **kubeflow-ragas-config.yaml** file as follows:

Example kubeflow-ragas-config.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kubeflow-ragas-config
  namespace: <project_name>
data:
  TRUSTYAI_EMBEDDING_MODEL: "all-MiniLM-L6-v2"
  KUBEFLOW_LLAMA_STACK_URL: "http://$<distribution_name>-
service.$<your_namespace>.svc.cluster.local:$<port>"
  KUBEFLOW_PIPELINES_ENDPOINT: "https://<kfp_endpoint>"
  KUBEFLOW_NAMESPACE: "<project_name>"
  KUBEFLOW_BASE_IMAGE: "registry.access.redhat.com/ubi9/python-312:latest"
  KUBEFLOW_RESULTS_S3_PREFIX: "s3://<bucket_name>/ragas-results"
  KUBEFLOW_S3_CREDENTIALS_SECRET_NAME: "<ragas_s3_credentials>"
```

- **TRUSTYAI_EMBEDDING_MODEL**: Used by Ragas for semantic similarity calculations.
- **KUBEFLOW_LLAMA_STACK_URL**: The URL for the Llama Stack server. This must be accessible from the Kubeflow Pipeline pods. The <distribution_name>, <namespace>, and <port> are replaced with the name of the LlamaStack distribution you are creating, the namespace where you are creating it, and the port. These 3 elements are present in the LlamaStack distribution YAML.
- **KUBEFLOW_PIPELINES_ENDPOINT**: The Kubeflow Pipelines API endpoint URL.
- **KUBEFLOW_NAMESPACE**: The namespace where pipeline runs are executed. This should match your current project namespace.
- **KUBEFLOW_BASE_IMAGE**: The base container image used to run the Ragas evaluation in the remote provider. Defaults to **registry.access.redhat.com/ubi9/python-312:latest**. The Kubeflow Pipeline components automatically install **llama-stack-provider-ragas[remote]** and its dependencies on top of this base image at runtime.
- **KUBEFLOW_RESULTS_S3_PREFIX**: The S3 path prefix where evaluation results are stored. For example: **s3://my-bucket/ragas-evaluation-results**.
- **KUBEFLOW_S3_CREDENTIALS_SECRET_NAME**: The name of the secret containing S3 credentials.

IMPORTANT

The Kubeflow Pipeline components automatically install **llama-stack-provider-ragas[remote]** at runtime. By default, this package is installed from the public Python Index (PyPI). No action is required for this default behavior.

If you are using a disconnected environment, the package is installed from the Red Hat Python Index. In this case, you must mirror the Python index as described in Mirror the Python Index for your disconnected environment .

9. Apply the ConfigMap:

```
$ oc apply -f kubeflow-ragas-config.yaml
```

10. Create a Llama Stack distribution configuration file with the Ragas remote provider. For example, create a llama-stack-ragas-remote.yaml as follows:

Example llama-stack-ragas-remote.yaml

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: llama-stack-pod
spec:
  replicas: 1
  server:
    containerSpec:
      resources:
        requests:
          cpu: 4
          memory: "12Gi"
        limits:
          cpu: 6
          memory: "14Gi"
      env:
        - name: INFERENCE_MODEL
          valueFrom:
            secretKeyRef:
              key: INFERENCE_MODEL
              name: llama-stack-inference-model-secret
              optional: true
        - name: VLLM_MAX_TOKENS
          value: "4096"
        - name: VLLM_URL
          valueFrom:
            secretKeyRef:
              key: VLLM_URL
              name: llama-stack-inference-model-secret
              optional: true
        - name: VLLM_TLS_VERIFY
          valueFrom:
            secretKeyRef:
              key: VLLM_TLS_VERIFY
              name: llama-stack-inference-model-secret
```

```
        optional: true
  - name: VLLM_API_TOKEN
    valueFrom:
      secretKeyRef:
        key: VLLM_API_TOKEN
        name: llama-stack-inference-model-secret
        optional: true
# Optional: only required when using inline Milvus Lite as a vector store.
# To use inline Milvus, also set ENABLE_INLINE_MILVUS to "true".
# Do not set these values when using remote Milvus, pgvector, or no vector store.
# - name: ENABLE_INLINE_MILVUS
#   value: "true"
# - name: MILVUS_DB_PATH
#   value: ~/.llama/milvus.db
  - name: FMS_ORCHESTRATOR_URL
    value: "http://localhost"
  - name: KUBEFLOW_PIPELINES_ENDPOINT
    valueFrom:
      configMapKeyRef:
        key: KUBEFLOW_PIPELINES_ENDPOINT
        name: kubeflow-ragas-config
        optional: true
  - name: KUBEFLOW_NAMESPACE
    valueFrom:
      configMapKeyRef:
        key: KUBEFLOW_NAMESPACE
        name: kubeflow-ragas-config
        optional: true
  - name: KUBEFLOW_BASE_IMAGE
    valueFrom:
      configMapKeyRef:
        key: KUBEFLOW_BASE_IMAGE
        name: kubeflow-ragas-config
        optional: true
  - name: KUBEFLOW_LLAMA_STACK_URL
    valueFrom:
      configMapKeyRef:
        key: KUBEFLOW_LLAMA_STACK_URL
        name: kubeflow-ragas-config
        optional: true
  - name: KUBEFLOW_RESULTS_S3_PREFIX
    valueFrom:
      configMapKeyRef:
        key: KUBEFLOW_RESULTS_S3_PREFIX
        name: kubeflow-ragas-config
        optional: true
  - name: KUBEFLOW_S3_CREDENTIALS_SECRET_NAME
    valueFrom:
      configMapKeyRef:
        key: KUBEFLOW_S3_CREDENTIALS_SECRET_NAME
        name: kubeflow-ragas-config
        optional: true
  - name: TRUSTYAI_EMBEDDING_MODEL
    valueFrom:
      configMapKeyRef:
        key: TRUSTYAI_EMBEDDING_MODEL
```

```
            name: kubeflow-ragas-config
            optional: true
        - name: KUBEFLOW_PIPELINES_TOKEN
          valueFrom:
            secretKeyRef:
              key: KUBEFLOW_PIPELINES_TOKEN
              name: kubeflow-pipelines-token
              optional: true
        - name: AWS_ACCESS_KEY_ID
          valueFrom:
            secretKeyRef:
              key: AWS_ACCESS_KEY_ID
              name: "<ragas_s3_credentials>"
              optional: true
        - name: AWS_SECRET_ACCESS_KEY
          valueFrom:
            secretKeyRef:
              key: AWS_SECRET_ACCESS_KEY
              name: "<ragas_s3_credentials>"
              optional: true
        - name: AWS_DEFAULT_REGION
          valueFrom:
            secretKeyRef:
              key: AWS_DEFAULT_REGION
              name: "<ragas_s3_credentials>"
              optional: true
      name: llama-stack
      port: 8321
    distribution:
      name: rh-dev
```

11. Deploy the Llama Stack distribution:

```
$ oc apply -f llama-stack-ragas-remote.yaml
```

12. Wait for the deployment to complete:

```
$ oc get pods -w
```

Wait until the **llama-stack-pod** pod status shows **Running**.

3.4. EVALUATING RAG SYSTEM QUALITY WITH RAGAS METRICS

Evaluate your RAG system quality by testing your setup, using the example provided in the demo notebook. This demo outlines the basic steps for evaluating your RAG system with Ragas using the Python client. You can execute the demo notebook steps from a Jupyter environment.

Alternatively, you can submit an evaluation by directly using the **http** methods of the Llama Stack API.

IMPORTANT

The Llama Stack pod must be accessible from the Jupyter environment in the cluster, which may not be the case by default. To configure this setup, see Ingesting content into a Llama model

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project.
- You have created a pipeline server.
- You have created a secret for your AWS credentials in your project namespace.
- You have deployed a Llama Stack distribution with the Ragas evaluation provider enabled (Inline or Remote). For more information, see Setting up the Ragas inline provider for development.
- You have access to a workbench or notebook environment where you can run Python code.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
2. Click the name of the project that contains the workbench.
3. Click the **Workbenches** tab.
4. If the status of the workbench is **Running**, skip to the next step.
   If the status of the workbench is **Stopped**, in the **Status** column for the workbench, click **Start**.

   The **Status** column changes from **Stopped** to **Starting** when the workbench server is starting, and then to **Running** when the workbench has successfully started.
5. Click the open icon () next to the workbench.
   Your Jupyter environment window opens.
6. On the toolbar, click the **Git Clone** icon and then select **Clone a Repository**.
7. In the **Clone a repo** dialog, enter the following URL **https://github.com/trustyai-explainability/llama-stack-provider-ragas.git**
8. In the file browser, select the newly-created **/llama-stack-provider-ragas/demos** folder.
   You see a Jupyter notebook named **basic_demo.ipynb**.
9. Double-click the **basic_demo.ipynb** file to launch the Jupyter notebook.
   The Jupyter notebook opens. You see code examples for the following tasks:
   - Run your Llama Stack distribution
   - Setup and Imports
   - Llama Stack Client Setup
   - Dataset Preparation
   - Dataset Registration
   - Benchmark Registration

- Evaluation Execution
- Inline vs Remote Side-by-side

10. In the Jupyter notebook, run the code cells sequentially through the **Evaluation Execution**.
11. Return to the OpenShift AI dashboard.
12. Click **Develop & train → Pipelines → Runs**. You might need to refresh the page to see that the new evaluation job running.
13. Wait for the job to show **Successful**.
14. Return to the workbench and run the **Results Display** cell.
15. Inspect the results displayed.

CHAPTER 4. USING LLAMA STACK WITH TRUSTYAI

This section contains tutorials for working with Llama Stack in TrustyAI. These tutorials demonstrate how to use various Llama Stack components and providers to evaluate and work with language models.

The following sections describe how to work with Llama Stack and provide example use cases:

- Using the Llama Stack external evaluation provider with lm-evaluation-harness in TrustyAI
- Running custom evaluations with LM-Eval Llama Stack external evaluation provider
- Using the trustyai-fms Guardrails Orchestrator with Llama Stack

4.1. USING LLAMA STACK EXTERNAL EVALUATION PROVIDER WITH LM-EVALUATION-HARNESS IN TRUSTYAI

This example demonstrates how to evaluate a language model in Red Hat OpenShift AI using the LMEval Llama Stack external eval provider in a Python workbench. To do this, configure a Llama Stack server to use the LMEval eval provider, register a benchmark dataset, and run a benchmark evaluation job on a language model.

Prerequisites

- You have installed Red Hat OpenShift AI, version 2.20 or later.
- You have cluster administrator privileges for your OpenShift AI cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have a large language model (LLM) for chat generation or text classification, or both, deployed in your namespace.
- You have installed TrustyAI Operator in your OpenShift AI cluster.
- You have set KServe to Raw Deployment mode in your cluster.

Procedure

1. Create and activate a Python virtual environment for this tutorial in your local machine:

```
python3 -m venv .venv
source .venv/bin/activate
```

2. Install the required packages from the Python Package Index (PyPI):

```
pip install \
    llama-stack \
    llama-stack-client \
    llama-stack-provider-lmeval
```

3. Create the model route:

```
oc create route edge vllm --service=<VLLM_SERVICE> --port=<VLLM_PORT> -n
<MODEL_NAMESPACE>
```

4. Configure the Llama Stack server. Set the variables to configure the runtime endpoint and namespace. The VLLM_URL value should be the **v1/completions** endpoint of your model route and the TRUSTYAI_LM_EVAL_NAMESPACE should be the namespace where your model is deployed. For example:

```
export TRUSTYAI_LM_EVAL_NAMESPACE=<MODEL_NAMESPACE>
export MODEL_ROUTE=$(oc get route -n "$TRUSTYAI_LM_EVAL_NAMESPACE" | awk
'/predictor/{print $2; exit}')
export VLLM_URL="https://${MODEL_ROUTE}/v1/completions"
```

5. Download the **providers.d** provider configuration directory and the **run.yaml** execution file:

```
curl --create-dirs --output providers.d/remote/eval/trustyai_lmeval.yaml
https://raw.githubusercontent.com/trustyai-explainability/llama-stack-provider-
lmeval/refs/heads/main/providers.d/remote/eval/trustyai_lmeval.yaml

curl --create-dirs --output run.yaml https://raw.githubusercontent.com/trustyai-
explainability/llama-stack-provider-lmeval/refs/heads/main/run.yaml
```

6. Start the Llama Stack server in a virtual environment, which uses port **8321** by default:

```
llama stack run run.yaml --image-type venv
```

7. Create a Python script in a Jupyter workbench and import the following libraries and modules, to interact with the server and run an evaluation:

```
import os
import subprocess

import logging

import time
import pprint
```

8. Start the Llama Stack Python client to interact with the running Llama Stack server:

```
BASE_URL = "http://localhost:8321"

def create_http_client():
    from llama_stack_client import LlamaStackClient
    return LlamaStackClient(base_url=BASE_URL)

client = create_http_client()
```

9. Print a list of the current available benchmarks:

```
benchmarks = client.benchmarks.list()

pprint.pprint(f"Available benchmarks: {benchmarks}")
```

10. LMEval provides access to over 100 preconfigured evaluation datasets. Register the ARC-Easy benchmark, a dataset of grade-school level, multiple-choice science questions:

```
client.benchmarks.register(
    benchmark_id="trustyai_lmeval::arc_easy",
    dataset_id="trustyai_lmeval::arc_easy",
    scoring_functions=["string"],
    provider_benchmark_id="string",
    provider_id="trustyai_lmeval",
     metadata={
        "tokenizer": "google/flan-t5-small",
        "tokenized_requests": False,
    }
)
```

11. Verify that the benchmark has been registered successfully:

```
benchmarks = client.benchmarks.list()
pprint.print(f"Available benchmarks: {benchmarks}")
```

12. Run a benchmark evaluation job on your deployed model using the following input. Replace phi-3 with the name of your deployed model:

```
job = client.eval.run_eval(
    benchmark_id="trustyai_lmeval::arc_easy",
    benchmark_config={
        "eval_candidate": {
            "type": "model",
            "model": "phi-3",
            "provider_id": "trustyai_lmeval",
            "sampling_params": {
                "temperature": 0.7,
                "top_p": 0.9,
                "max_tokens": 256
            },
        },
        "num_examples": 1000,
    },
)

print(f"Starting job '{job.job_id}'")
```

13. Monitor the status of the evaluation job using the following code. The job will run asynchronously, so you can check its status periodically:

```
def get_job_status(job_id, benchmark_id):
    return client.eval.jobs.status(job_id=job_id, benchmark_id=benchmark_id)

while True:
    job = get_job_status(job_id=job.job_id, benchmark_id="trustyai_lmeval::arc_easy")
```

```
print(job)

if job.status in ['failed', 'completed']:
    print(f"Job ended with status: {job.status}")
    break

time.sleep(20)
```

1. Retrieve the evaluation job results once the job status reports back as **completed**:

```
pprint.pprint(client.eval.jobs.retrieve(job_id=job.job_id,
benchmark_id="trustyai_lmeval::arc_easy").scores)
```

4.2. RUNNING CUSTOM EVALUATIONS WITH LM-EVAL AND LLAMA STACK

This example demonstrates how to use the LM-Eval Llama Stack external eval provider to evaluate a language model with a custom benchmark. Creating a custom benchmark is useful for evaluating specific model knowledge and behavior.

The process involves three steps:

- Uploading the task dataset to your OpenShift AI cluster
- Registering it as a custom benchmark dataset with Llama Stack
- Running a benchmark evaluation job on a language model

Prerequisites

- You have installed Red Hat OpenShift AI, version 2.20 or later.
- You have cluster administrator privileges for your OpenShift AI cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have a large language model (LLM) for chat generation or text classification, or both, deployed on vLLM Serving Runtime in your OpenShift AI cluster.
- You have installed TrustyAI Operator in your OpenShift AI cluster.
- You have set KServe to Raw Deployment mode in your cluster.

Procedure

1. Upload your custom benchmark dataset to your OpenShift cluster using a PersistentVolumeClaim (PVC) and a temporary pod. Create a PVC named **my-pvc** to store your dataset. Run the following command in your CLI, replacing <MODEL_NAMESPACE> with the namespace of your language model:

```
oc apply -n <MODEL_NAMESPACE> -f - << EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
   name: my-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
EOF
```

2. Create a pod object named **dataset-storage-pod** to download the task dataset into the PVC. This pod is used to copy your dataset from your local machine to the OpenShift AI cluster:

```
oc apply -n <MODEL_NAMESPACE> -f - << EOF
apiVersion: v1
kind: Pod
metadata:
  name: dataset-storage-pod
spec:
  containers:
  - name: dataset-container
    image: 'quay.io/prometheus/busybox:latest'
    command: ["/bin/sh", "-c", "sleep 3600"]
    volumeMounts:
    - mountPath: "/data/upload_files"
      name: dataset-storage
  volumes:
  - name: dataset-storage
    persistentVolumeClaim:
      claimName: my-pvc
EOF
```

3. Copy your locally stored task dataset to the pod to place it within the PVC. In this example, the dataset is named **example-dk-bench-input-bmo.jsonl** locally and it is copied to the **dataset-storage-pod** under the path **/data/upload_files/**.

```
oc cp example-dk-bench-input-bmo.jsonl dataset-storage-pod:/data/upload_files/example-dk-bench-input-bmo.jsonl -n <MODEL_NAMESPACE>
```

4. Once the custom dataset is uploaded to the PVC, register it as a benchmark for evaluations. At a minimum, provide the following metadata and replace the **DK_BENCH_DATASET_PATH** and any other metadata fields to match your specific configuration:

   a. The TrustyAI LM-Eval Tasks GitHub web address

   b. Your branch

   c. The commit hash and path of the custom task.

```
client.benchmarks.register(
    benchmark_id="trustyai_lmeval::dk-bench",
    dataset_id="trustyai_lmeval::dk-bench",
```

```
    scoring_functions=["accuracy"],
    provider_benchmark_id="dk-bench",
    provider_id="trustyai_lmeval",
    metadata={
       "custom_task": {
          "git": {
             "url": "https://github.com/trustyai-explainability/lm-eval-tasks.git",
             "branch": "main",
             "commit": "8220e2d73c187471acbe71659c98bccecfe77958",
             "path": "tasks/",
          }
       },
       "env": {
          # Path of the dataset inside the PVC
          "DK_BENCH_DATASET_PATH": "/opt/app-root/src/hf_home/example-dk-bench-
 input-bmo.jsonl",
          "JUDGE_MODEL_URL": "http://phi-3-predictor:8080/v1/chat/completions",
          # For simplicity, we use the same model as the one being evaluated
          "JUDGE_MODEL_NAME": "phi-3",
          "JUDGE_API_KEY": "",
       },
       "tokenized_requests": False,
       "tokenizer": "google/flan-t5-small",
       "input": {"storage": {"pvc": "my-pvc"}}
    },
)
```

5. Run a benchmark evaluation on your model:

```
job = client.eval.run_eval(
    benchmark_id="trustyai_lmeval::dk-bench",
    benchmark_config={
       "eval_candidate": {
          "type": "model",
          "model": "phi-3",
          "provider_id": "trustyai_lmeval",
          "sampling_params": {
             "temperature": 0.7,
             "top_p": 0.9,
             "max_tokens": 256
          },
       },
       "num_examples": 1000,
    },
)

print(f"Starting job '{job.job_id}'")
```

6. Monitor the status of the evaluation job. The job runs asynchronously, so you can check its status periodically:

```
import time
def get_job_status(job_id, benchmark_id):
    return client.eval.jobs.status(job_id=job_id, benchmark_id=benchmark_id)

while True:
```

```
job = get_job_status(job_id=job.job_id, benchmark_id="trustyai_lmeval::dk-bench")
print(job)

if job.status in ['failed', 'completed']:
    print(f"Job ended with status: {job.status}")
    break

time.sleep(20)
```

4.3. DETECTING PERSONALLY IDENTIFIABLE INFORMATION (PII) BY USING GUARDRAILS WITH LLAMA STACK

The **trustyai_fms** Orchestrator server is an external provider for Llama Stack that allows you to configure and use the Guardrails Orchestrator and compatible detection models through the Llama Stack API. This implementation of Llama Stack combines Guardrails Orchestrator with a suite of community-developed detectors to provide robust content filtering and safety monitoring. Guardrails execution is independent of the configured vector store and does not require Milvus or pgvector to be enabled.

This example demonstrates how to use the built-in Guardrails Regex Detector to detect personally identifiable information (PII) with Guardrails Orchestrator as Llama Stack safety guardrails, using the **LlamaStack** Operator to deploy a distribution in your Red Hat OpenShift AI namespace.

NOTE

Guardrails Orchestrator with Llama Stack is not supported on **s390x**, as it requires the LlamaStack Operator, which is currently unavailable for this architecture.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have a large language model (LLM) for chat generation or text classification, or both, deployed in your namespace.
- You have configured the **spec.kserve.rawDeploymentServiceConfig** field to **Headed** in your **DataScienceCluster**.
- A cluster administrator has installed the following Operators in OpenShift:
  - Red Hat Connectivity Link version 1.1.1 or later.

NOTE

You must uninstall OpenShift Service Mesh, version 2.6.7-0 or later, from your cluster.

Procedure

1. Configure your OpenShift AI environment with the following configurations in the **DataScienceCluster**. Note that you must manually update the **spec.llamastack.managementState** field to **Managed**:

```
spec:
  trustyai:
    managementState: Managed
  llamastack:
    managementState: Managed
  kserve:
    defaultDeploymentMode: RawDeployment
    managementState: Managed
    nim:
      managementState: Managed
    rawDeploymentServiceConfig: Headed
  serving:
    ingressGateway:
      certificate:
        type: OpenshiftDefaultIngress
    managementState: Removed
    name: knative-serving
  serviceMesh:
    managementState: Removed
```

2. Create a project in your OpenShift AI namespace:

```
PROJECT_NAME="lls-minimal-example"
oc new-project $PROJECT_NAME
```

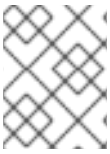
3. Deploy the Guardrails Orchestrator with regex detectors by applying the Orchestrator configuration for regex-based PII detection:

```
cat <<EOF | oc apply -f -
kind: ConfigMap
apiVersion: v1
metadata:
  name: fms-orchestr8-config-nlp
data:
  config.yaml: |
    detectors:
      regex:
        type: text_contents
        service:
          hostname: "127.0.0.1"
          port: 8080
        chunker_id: whole_doc_chunker
        default_threshold: 0.5
---
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: GuardrailsOrchestrator
metadata:
  name: guardrails-orchestrator
spec:
  orchestratorConfig: "fms-orchestr8-config-nlp"
  enableBuiltInDetectors: true
```

```
  enableGuardrailsGateway: false
  replicas: 1
EOF
```

4. In the same namespace, create a Llama Stack distribution:

```
apiVersion: llamastack.io/v1alpha1
kind: LlamaStackDistribution
metadata:
  name: llamastackdistribution-sample
  namespace: <PROJECT_NAMESPACE>
spec:
  replicas: 1
  server:
    containerSpec:
      env:
        - name: VLLM_URL
          value: '${VLLM_URL}'
        - name: INFERENCE_MODEL
          value: '${INFERENCE_MODEL}'
      # Optional: only required when using inline Milvus Lite as a vector store.
      # To use inline Milvus, also set ENABLE_INLINE_MILVUS to "true".
      # Do not set these values when using remote Milvus, pgvector, or no vector store.
      # - name: ENABLE_INLINE_MILVUS
      #   value: "true"
      # - name: MILVUS_DB_PATH
      #   value: ~/.llama/milvus.db
        - name: VLLM_TLS_VERIFY
          value: 'false'
        - name: FMS_ORCHESTRATOR_URL
          value: '${FMS_ORCHESTRATOR_URL}'
    name: llama-stack
    port: 8321
  distribution:
    name: rh-dev
  storage:
    size: 20Gi
```



NOTE

– After deploying the **LlamaStackDistribution** CR, a new pod is created in the same namespace. This pod runs the LlamaStack server for your distribution. –

5. Once the Llama Stack server is running, use the **/v1/shields** endpoint to dynamically register a shield. For example, register a shield that uses regex patterns to detect personally identifiable information (PII).

6. Open a port-forward to access it locally:

```
oc -n $PROJECT_NAME port-forward svc/llama-stack 8321:8321
```

7. Use the **/v1/shields** endpoint to dynamically register a shield. For example, register a shield that uses regex patterns to detect personally identifiable information (PII):

```
curl -X POST http://localhost:8321/v1/shields \
  -H 'Content-Type: application/json' \
  -d '{
    "shield_id": "regex_detector",
    "provider_shield_id": "regex_detector",
    "provider_id": "trustyai_fms",
    "params": {
      "type": "content",
      "confidence_threshold": 0.5,
      "message_types": ["system", "user"],
      "detectors": {
        "regex": {
          "detector_params": {
            "regex": ["email", "us-social-security-number", "credit-card"]
          }
        }
      }
    }
  }'
```

8. Verify that the shield was registered:

```
curl -s http://localhost:8321/v1/shields | jq '.'
```

9. The following output indicates that the shield has been registered successfully:

```
{
  "data": [
    {
      "identifier": "regex_detector",
      "provider_resource_id": "regex_detector",
      "provider_id": "trustyai_fms",
      "type": "shield",
      "params": {
        "type": "content",
        "confidence_threshold": 0.5,
        "message_types": [
          "system",
          "user"
        ],
        "detectors": {
          "regex": {
            "detector_params": {
              "regex": [
                "email",
                "us-social-security-number",
                "credit-card"
              ]
            }
          }
        }
      }
    }
  ]
}
```

10. Once the shield has been registered, verify that it is working by sending a message containing PII to the **/v1/safety/run-shield** endpoint:

    a. Email detection example:

```
curl -X POST http://localhost:8321/v1/safety/run-shield \
-H "Content-Type: application/json" \
-d '{
  "shield_id": "regex_detector",
  "messages": [
    {
      "content": "My email is test@example.com",
      "role": "user"
    }
  ]
}' | jq '.'
```

    This should return a response indicating that the email was detected:

```
{
  "violation": {
    "violation_level": "error",
    "user_message": "Content violation detected by shield regex_detector (confidence:
1.00, 1/1 processed messages violated)",
    "metadata": {
      "status": "violation",
      "shield_id": "regex_detector",
      "confidence_threshold": 0.5,
      "summary": {
        "total_messages": 1,
        "processed_messages": 1,
        "skipped_messages": 0,
        "messages_with_violations": 1,
        "messages_passed": 0,
        "message_fail_rate": 1.0,
        "message_pass_rate": 0.0,
        "total_detections": 1,
        "detector_breakdown": {
          "active_detectors": 1,
          "total_checks_performed": 1,
          "total_violations_found": 1,
          "violations_per_message": 1.0
        }
      },
      "results": [
        {
          "message_index": 0,
          "text": "My email is test@example.com",
          "status": "violation",
          "score": 1.0,
          "detection_type": "pii",
          "individual_detector_results": [
            {
              "detector_id": "regex",
              "status": "violation",
              "score": 1.0,
```

```
            "detection_type": "pii"
          }
        ]
      }
    ]
  }
}
```

b. Social security number (SSN) detection example:

```
curl -X POST http://localhost:8321/v1/safety/run-shield \
-H "Content-Type: application/json" \
-d '{
    "shield_id": "regex_detector",
    "messages": [
      {
        "content": "My SSN is 123-45-6789",
        "role": "user"
      }
    ]
}' | jq '.'
```

This should return a response indicating that the SSN was detected:

```
{
  "violation": {
    "violation_level": "error",
    "user_message": "Content violation detected by shield regex_detector (confidence:
1.00, 1/1 processed messages violated)",
    "metadata": {
      "status": "violation",
      "shield_id": "regex_detector",
      "confidence_threshold": 0.5,
      "summary": {
        "total_messages": 1,
        "processed_messages": 1,
        "skipped_messages": 0,
        "messages_with_violations": 1,
        "messages_passed": 0,
        "message_fail_rate": 1.0,
        "message_pass_rate": 0.0,
        "total_detections": 1,
        "detector_breakdown": {
          "active_detectors": 1,
          "total_checks_performed": 1,
          "total_violations_found": 1,
          "violations_per_message": 1.0
        }
      },
      "results": [
        {
          "message_index": 0,
          "text": "My SSN is 123-45-6789",
          "status": "violation",
          "score": 1.0,
```

```
          "detection_type": "pii",
          "individual_detector_results": [
           {
            "detector_id": "regex",
            "status": "violation",
            "score": 1.0,
            "detection_type": "pii"
           }
          ]
         }
        ]
       }
      }
}
```

c. Credit card detection example:

```
curl -X POST http://localhost:8321/v1/safety/run-shield \
-H "Content-Type: application/json" \
-d '{
    "shield_id": "regex_detector",
    "messages": [
      {
        "content": "My credit card number is 4111-1111-1111-1111",
        "role": "user"
      }
    ]
}' | jq '.'
```

This should return a response indicating that the credit card number was detected:

```
{
  "violation": {
    "violation_level": "error",
    "user_message": "Content violation detected by shield regex_detector (confidence:
1.00, 1/1 processed messages violated)",
    "metadata": {
      "status": "violation",
      "shield_id": "regex_detector",
      "confidence_threshold": 0.5,
      "summary": {
        "total_messages": 1,
        "processed_messages": 1,
        "skipped_messages": 0,
        "messages_with_violations": 1,
        "messages_passed": 0,
        "message_fail_rate": 1.0,
        "message_pass_rate": 0.0,
        "total_detections": 1,
        "detector_breakdown": {
          "active_detectors": 1,
          "total_checks_performed": 1,
          "total_violations_found": 1,
          "violations_per_message": 1.0
        }
```

```
    },
    "results": [
     {
      "message_index": 0,
      "text": "My credit card number is 4111-1111-1111-1111",
      "status": "violation",
      "score": 1.0,
      "detection_type": "pii",
      "individual_detector_results": [
       {
        "detector_id": "regex",
        "status": "violation",
        "score": 1.0,
        "detection_type": "pii"
       }
      ]
     }
    ]
   }
  }
}
```

CHAPTER 5. EVALUATE LLMS WITH EVALHUB

Use EvalHub to evaluate your large language models against standardized benchmarks, track results with MLflow, and manage evaluation workflows across multiple tenants.

5.1. UNDERSTANDING EVALHUB

EvalHub is an evaluation orchestration service for large language models (LLMs) on Red Hat OpenShift AI. It provides a versioned REST API for submitting evaluation jobs, managing benchmark providers, and tracking results through MLflow experiment tracking. Each evaluation runs as an isolated Job, enabling parallel execution and horizontal scalability across namespaces and tenants.

EvalHub consists of three components:

- **EvalHub Server** – A REST API service that handles evaluation workflows, job orchestration, and provider management, with PostgreSQL storage.
- **EvalHub SDK and CLI** – A Python client library and command-line tool for submitting evaluations and building framework adapters. The CLI provides the **evalhub** command for interacting with EvalHub from the terminal.
- **Providers** – Evaluation framework adapters packaged as container images. Each provider translates EvalHub job requests into evaluation framework-specific commands and reports results back to the server.

5.1.1. Core concepts

The following concepts are central to EvalHub.

Providers

A provider represents an evaluation framework, such as **lm_evaluation_harness**, **garak**, **guidellm**, or **lighteval**. Each provider includes a set of benchmarks. EvalHub includes built-in providers that are read-only.

Benchmarks

A benchmark is a specific evaluation task within a provider. For example, the **lm_evaluation_harness** provider includes benchmarks such as **mmlu**, **hellaswag**, **arc_challenge**, and **gsm8k**. Each benchmark has a category such as **math**, **reasoning**, **safety**, or **code**, along with metrics and optional pass criteria.

Collections

A collection groups benchmarks from one or more providers into a reusable evaluation suite. For example, a **safety-and-fairness-v1** collection might combine safety benchmarks from **lm_evaluation_harness** with vulnerability scans from **garak**.

Pass criteria and thresholds

Pass criteria define the minimum score that a benchmark or job must achieve to pass. Thresholds can be set at three levels, from most to least specific:

1. **Benchmark level** – You set a benchmark-level threshold per benchmark in a job submission or collection definition. This overrides all other thresholds.
2. **Collection level** – A collection-level threshold applies to all benchmarks in the collection that do not have their own threshold.

3. **Provider level** – A provider-level threshold is the default threshold defined in the provider's benchmark configuration.
   Each benchmark declares a **primary score** metric, such as **acc_norm** or **toxicity_score**, and optionally a **lower_is_better** flag. When **lower_is_better** is **false** (the default), the benchmark passes if the score is greater than or equal to the threshold. When **lower_is_better** is **true**, it passes if the score is less than or equal to the threshold.

   Each benchmark in a collection or job can be assigned a weight that controls its relative importance in the overall score. At the job level, EvalHub computes a weighted average of all benchmark primary scores and compares it against the job-level threshold to determine an overall pass or fail result.

Evaluation jobs

An evaluation job represents a single evaluation run against a model. A job references either a list of benchmarks or a collection, a model endpoint, and optional MLflow experiment configuration. Jobs progress through states: **pending**, **running**, **completed**, **failed**, **cancelled**, or **partially_failed**.

Adapters

An adapter wraps an evaluation framework, such as **lm_evaluation_harness**, and implements the **FrameworkAdapter** interface so that EvalHub can orchestrate the evaluation. Adapters are packaged as Red Hat Universal Base Image 9 (UBI9) container images.

5.2. EVALHUB ARCHITECTURE OVERVIEW

When you submit an evaluation job, EvalHub follows this workflow:

1. The client submits a job through the REST API or SDK.
2. The server validates the request, resolves benchmarks, and persists the job with a status of **pending**.
3. The runtime creates a Kubernetes Job for each benchmark. Each Job pod contains two containers:
   - The **adapter** container runs the evaluation framework. Adapters are provider-specific container images that implement a standard interface, translating the job specification into the evaluation framework-specific invocations and returning structured results.
   - The **sidecar proxy** container authenticates to the EvalHub server using a **ServiceAccount** token and forwards status events and results from the adapter. The sidecar also proxies authenticated requests to MLflow and OCI registries when configured. This design keeps credentials out of the adapter container, which can run custom user-provided code.
4. The adapter runs the evaluation and reports status events back to EvalHub through the sidecar.
5. The server aggregates and stores the results. If MLflow integration is enabled, the server also logs the results to MLflow.

5.3. DEPLOY EVALHUB WITH THE TRUSTYAI OPERATOR

Deploy EvalHub through the TrustyAI Operator as part of the OpenShift AI.

Prerequisites

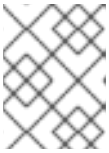
- You have cluster administrator privileges for your OpenShift cluster.

- You have installed the OpenShift CLI (**oc**) version 4.12 or later.
- You have the TrustyAI component in your OpenShift AI **DataScienceCluster** set to **Managed**.
- You have configured KServe to use **RawDeployment** mode.

Procedure

1. Create a Secret containing the PostgreSQL connection string. The Secret must contain a **db-url** key with a valid PostgreSQL connection URI:

```
apiVersion: v1
kind: Secret
metadata:
  name: evalhub-db-credentials
type: Opaque
stringData:
  db-url: "postgres://evalhub:changeme@postgresql.evalhub.svc.cluster.local:5432/evalhub"
```



NOTE

Replace the hostname, credentials including the **changeme** placeholder, and database name to match your PostgreSQL deployment.

```
$ oc apply -f evalhub-db-credentials.yaml -n <namespace>
```

2. Create an EvalHub custom resource to deploy the service:

Example evalhub_cr.yaml

```
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: EvalHub
metadata:
  name: evalhub
spec:
  replicas: 1
  database:
    type: postgresql
    secret: evalhub-db-credentials
  providers:
    - lm_evaluation_harness
    - garak
    - guidellm
  collections:
    - safety-and-fairness-v1
  env:
    - name: MLFLOW_TRACKING_URI
      value: "http://mlflow.mlflow.svc.cluster.local:5000"
```

Table 5.1. EvalHub custom resource parameters

Parameter	Description
replicas	The number of EvalHub pods to create.
database.type	Storage backend. Set to **postgresql** for PostgreSQL.
database.secret	Name of a Secret containing the PostgreSQL connection string.
providers	List of evaluation provider configurations to load at startup.
collections	List of benchmark collections to load at startup.
otel	Optional: OpenTelemetry exporter configuration for traces and metrics.
env	Environment variables to set in the EvalHub deployment containers.

3. Apply the custom resource to the cluster:

```
$ oc apply -f evalhub_cr.yaml -n <namespace>
```

NOTE

Use a dedicated namespace for EvalHub rather than **redhat-ods-applications**. The **redhat-ods-applications** namespace has NetworkPolicies that restrict cross-namespace traffic, which requires additional labeling on tenant namespaces. For more information, see Section 5.23, "Set up a tenant namespace".

The TrustyAI Operator automatically reconciles the EvalHub custom resource in your namespace.

Verification

1. Confirm that the EvalHub pod is running:

```
$ oc get pods -l app=eval-hub -n <namespace>
```

Example output

```
NAME                      READY   STATUS    RESTARTS   AGE
evalhub-7b9f4c6d88-x2k4p  1/1     Running   0          2m
```

2. Query the health endpoint:

```
$ export EVALHUB_URL=https://$(oc get routes evalhub -o jsonpath='{.spec.host}' -n
<namespace>)
$ curl $EVALHUB_URL/api/v1/health | jq .
```

Example response

```
{
  "status": "healthy",
  "timestamp": "2026-04-13T10:00:00Z",
  "version": "0.3.0",
  "uptime": 3600000000000,
  "active_evaluations": 0
}
```

3. Install the EvalHub Python SDK to interact with the server. To install the SDK client library, run the following command:

```
$ pip install "eval-hub-sdk[client]"
```

To also include the CLI, run the following command:

```
$ pip install "eval-hub-sdk[cli]"
```

5.4. EVALHUB MULTI-TENANCY

EvalHub is a multi-tenant service. All API requests, except requests to **/api/v1/health**, must include the **X-Tenant** header, which identifies the target namespace. Resources such as jobs, providers, and collections are scoped to the tenant specified in this header. For information about setting up tenant namespaces and granting access, see Section 5.22, "EvalHub multi-tenancy and RBAC".

When using **curl**, include the **-H "X-Tenant: <namespace>"** header in each request.

When using the Python SDK, set the tenant at client initialization:

```
from evalhub import SyncEvalHubClient

client = SyncEvalHubClient(
    base_url="https://evalhub.example.com",
    tenant="my-namespace"
)
```

When using the CLI, configure the tenant in your connection profile. The CLI stores connection settings in named profiles at **~/.config/evalhub/config.yaml**. Settings are persistent across commands. Use **--profile <name>** to override the active profile at runtime.

```
$ evalhub config set tenant my-namespace
```

All API requests must also include an **Authorization: Bearer $TOKEN** header. The **curl** examples in this guide assume you have stored the EvalHub route URL in the **EVALHUB_URL** environment variable and a valid bearer token in the **TOKEN** environment variable. For information about obtaining the route URL, see Section 5.3, "Deploy EvalHub with the TrustyAI Operator". For information about obtaining a bearer token, see Section 5.24, "Grant access to EvalHub".

5.5. LIST EVALHUB PROVIDERS AND BENCHMARKS

List the evaluation providers and benchmarks registered in EvalHub to see which evaluation frameworks and tasks are available for your jobs. You can list providers using the REST API, Python SDK, or CLI.

Prerequisites

- You have a running EvalHub instance.

Procedure

1. List all registered providers:

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>"
$EVALHUB_URL/api/v1/evaluations/providers | jq .
```

Example output

```
{
  "items": [
    {
      "resource": { "id": "lm_evaluation_harness", "owner": "system" },
      "name": "lm_evaluation_harness",
      "title": "LM Evaluation Harness",
      "benchmarks": [ ... ]
    },
    {
      "resource": { "id": "garak", "owner": "system" },
      "name": "garak",
      "title": "Garak",
      "benchmarks": [ ... ]
    }
  ]
}
```

2. Get a specific provider with its benchmarks:

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>"
$EVALHUB_URL/api/v1/evaluations/providers/lm_evaluation_harness | jq .
```

Example output

```
{
  "resource": { "id": "lm_evaluation_harness", "owner": "system" },
  "name": "lm_evaluation_harness",
  "title": "LM Evaluation Harness",
  "benchmarks": [
    { "id": "mmlu", "name": "MMLU", "category": "reasoning" },
    { "id": "hellaswag", "name": "HellaSwag", "category": "reasoning" },
    { "id": "arc_challenge", "name": "ARC Challenge", "category": "reasoning" },
    ...
  ]
}
```

Alternatively, use the Python SDK:

```
from evalhub.client import SyncEvalHubClient

client = SyncEvalHubClient(
```

```
    base_url="https://evalhub.example.com",
    tenant="my-namespace"
)

for provider in client.providers.list():
    print(f"{provider.resource.id}: {provider.name}")

benchmarks = client.benchmarks.list(provider_id="lm_evaluation_harness")
for b in benchmarks:
    print(f"  {b.id}: {b.name}")
```

+ .Example output

```
lm_evaluation_harness: LM Evaluation Harness
garak: Garak
guidellm: GuideLLM
  mmlu: Massive Multitask Language Understanding
  hellaswag: HellaSwag
  gsm8k: Grade School Math 8K
  ...
```

Alternatively, use the CLI:

```
$ evalhub providers list
```

+ .Example output

```
 ID                  NAME                 DESCRIPTION                          BENCHMARKS
 lm_evaluation_harness  LM Evaluation Harness  EleutherAI language model evaluation     167
 garak               Garak                LLM vulnerability and safety scanner     12
 guidellm            GuideLLM             Performance benchmarking                  4
```

To get details for a specific provider:

```
$ evalhub providers describe lm_evaluation_harness
```

+ .Example output

```
Provider: LM Evaluation Harness
ID:       lm_evaluation_harness
Description: EleutherAI language model evaluation framework

Benchmarks (167):
 ID            NAME                          CATEGORY            METRICS
 mmlu          Massive Multitask Language Und…   knowledge           acc, acc_norm
 hellaswag     HellaSwag                     reasoning           acc, acc_norm
 gsm8k         Grade School Math 8K              math                exact_match
 arc_easy      ARC Easy                      reasoning           acc, acc_norm
 ...
```

Verification

- Confirm that the provider list is not empty and includes the built-in providers enabled in your EvalHub deployment.

5.6. SUBMIT AN EVALUATION JOB

Submit an evaluation job in EvalHub by specifying a model endpoint and one or more benchmarks. EvalHub runs the benchmarks against the model and returns a job ID that you can use to track results.

Prerequisites

- You have a running EvalHub instance.
- You have a model endpoint accessible from within the cluster.
- You know which providers and benchmarks are available. See Section 5.5, "List EvalHub providers and benchmarks".

Procedure

1. Submit a job by specifying the model endpoint and one or more benchmarks:

```
$ curl -X POST $EVALHUB_URL/api/v1/evaluations/jobs \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "X-Tenant: <namespace>" \
  -d '{
    "model": {
      "url": "http://my-model.my-namespace.svc.cluster.local:8080/v1",
      "name": "my-model"
    },
    "benchmarks": [
      {
        "provider_id": "lm_evaluation_harness",
        "benchmark_id": "mmlu"
      },
      {
        "provider_id": "lm_evaluation_harness",
        "benchmark_id": "hellaswag"
      }
    ]
  }'
```

NOTE

Most providers expect the model URL to point to an OpenAI-compatible inference endpoint. The required URL format may vary depending on the provider. Check the provider documentation for specific requirements.

The server returns a **202 Accepted** response with the job resource, including a job ID for tracking.

Alternatively, use the Python SDK:

```
from evalhub.client import SyncEvalHubClient
from evalhub.models import JobSubmissionRequest, ModelConfig, BenchmarkConfig

client = SyncEvalHubClient(
```

```
    base_url="https://evalhub.example.com",
    tenant="my-namespace"
)

job = client.jobs.create(JobSubmissionRequest(
    model=ModelConfig(
        url="http://my-model.my-namespace.svc.cluster.local:8080/v1",
        name="my-model"
    ),
    benchmarks=[
        BenchmarkConfig(provider_id="lm_evaluation_harness", benchmark_id="mmlu"),
        BenchmarkConfig(provider_id="lm_evaluation_harness", benchmark_id="hellaswag"),
    ]
))

print(f"Job ID: {job.resource.id}")
```

Alternatively, use the CLI:

```
$ evalhub eval run \
    --name my-eval \
    --model-url http://my-model.my-namespace.svc.cluster.local:8080/v1 \
    --model-name my-model \
    --provider lm_evaluation_harness \
    -b mmlu -b hellaswag
```

You can also submit from a YAML config file:

```
$ evalhub eval run --config evaljob.yaml
```

Verification

- Confirm the job is registered and check its status:

  ```
  $ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>" \
      $EVALHUB_URL/api/v1/evaluations/jobs/<job_id> | jq .status.state
  ```

  The job status transitions from **pending** to **running** to **completed**.

  Alternatively, use the CLI:

  ```
  $ evalhub eval status <job_id>
  ```

  Alternatively, use the Python SDK:

  ```
  job = client.jobs.get(job_id)
  print(job.state)
  ```

5.7. TRACK EVALUATION JOBS AND RESULTS

Track the status of running evaluation jobs and retrieve results after completion. You can check individual jobs, list all jobs, and filter by status.

Prerequisites

- You have submitted an evaluation job to EvalHub.
- You have the job ID returned from the submission.

Procedure

1. Check the status of a specific job:

```
$ curl -s \
    -H "Authorization: Bearer $TOKEN" \
    -H "X-Tenant: <namespace>" \
    $EVALHUB_URL/api/v1/evaluations/jobs/<job_id> | jq .
```

Example response for a completed job


```
{
  "resource": {
    "id": "<job_id>",
    "tenant": "<namespace>",
    "created_at": "2026-04-22T10:00:00Z"
  },
  "status": {
    "state": "completed",
    "benchmarks": [
      { "id": "mmlu", "provider_id": "lm_evaluation_harness", "status": "completed" },
      { "id": "hellaswag", "provider_id": "lm_evaluation_harness", "status": "completed" }
    ]
  },
  "results": {
    "benchmarks": [
      {
        "id": "mmlu",
        "provider_id": "lm_evaluation_harness",
        "metrics": { "acc": 0.65, "acc_norm": 0.68 }
      },
      {
        "id": "hellaswag",
        "provider_id": "lm_evaluation_harness",
        "metrics": { "acc": 0.72, "acc_norm": 0.75 }
      }
    ]
  },
  "name": "my-eval",
  "model": {
    "url": "http://my-model:8080/v1",
    "name": "my-model"
  },
  ...
}
```


2. After the job completes, retrieve the benchmark results:

```
$ curl -s \
```

```
    -H "Authorization: Bearer $TOKEN" \
    -H "X-Tenant: <namespace>" \
    $EVALHUB_URL/api/v1/evaluations/jobs/<job_id> | jq .results
```

The **results** object contains benchmark scores, metrics, and pass/fail outcomes. If pass criteria are configured, the results include a **test** field with the overall score, threshold, and pass/fail status.

3. List all jobs, optionally filtered by status:

```
$ curl -s \
    -H "Authorization: Bearer $TOKEN" \
    -H "X-Tenant: <namespace>" \
    "$EVALHUB_URL/api/v1/evaluations/jobs?status=completed&limit=10" | jq .
```

Table 5.2. Job query parameters

Parameter	Default	Description
limit	**50**	Maximum number of results to return. The maximum allowed value is 100.
offset	**0**	Number of results to skip for pagination.
status	–	Filter by job state: **pending**, **running**, **completed**, **failed**, **cancelled**, **partially_failed**.
name	–	Filter by job name. Uses exact, case-sensitive matching.
tags	–	Filter by a single tag. Returns jobs that contain the specified tag in their tags list.
owner	–	Filter by the authenticated username of the job owner, for example **system:serviceaccount:<namespace>:<name>** for a **ServiceAccount** or the OpenShift username.
experiment_id	–	Filter by MLflow experiment ID.

Alternatively, use the CLI.

To watch a job's status in real time, use the **--watch** flag. The CLI polls the job at regular intervals and displays benchmark progress until the job reaches a terminal state:

```
$ evalhub eval status --watch <job_id>
```

To retrieve formatted results after a job completes:

```
$ evalhub eval results <job_id> --format table
```

+ .Example output

```
BENCHMARK   PROVIDER              METRIC     VALUE
mmlu        lm_evaluation_harness   acc        0.65
mmlu        lm_evaluation_harness   acc_norm   0.68
hellaswag   lm_evaluation_harness   acc        0.72
hellaswag   lm_evaluation_harness   acc_norm   0.75
```

The **--format** flag supports **table**, **json**, **yaml**, and **csv**.

Alternatively, use the Python SDK.

To check the status of a specific job:

```
job = client.jobs.get(job_id)
print(f"State: {job.state}")
```

To wait for a job to complete:

```
result = client.jobs.wait_for_completion(job_id, timeout=3600, poll_interval=5.0)
for b in result.results.benchmarks:
    print(f"{b.id}: {b.metrics}")
```

To list jobs filtered by status:

```
from evalhub.models import JobStatus

completed_jobs = client.jobs.list(status=JobStatus.COMPLETED, limit=10)
for job in completed_jobs:
    print(f"{job.id}: {job.state}")
```

5.8. CANCEL AND DELETE JOBS

Cancel a running evaluation job or permanently delete a job record from the database.

Prerequisites

- You have submitted an evaluation job to EvalHub.
- You have the job ID of the job to cancel or delete.
- You have **delete** permissions on the **evaluations** virtual resource in the tenant namespace. For more information, see Section 5.24, “Grant access to EvalHub” .

Procedure

Run one of the following commands depending on whether you want to cancel or permanently delete the job:

1. To cancel a running job with a soft delete, where the job is marked as **cancelled** but the record is preserved for auditing, run the following command:

```
$ curl -X DELETE -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>"
$EVALHUB_URL/api/v1/evaluations/jobs/<job_id>
```

2. To permanently delete a job record from the database, run the following command with the **hard_delete** query parameter:

WARNING

The **hard_delete** operation permanently removes the job record from the database. This action cannot be undone, and the job results will no longer be available for auditing.

```
$ curl -X DELETE -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>"
"$EVALHUB_URL/api/v1/evaluations/jobs/<job_id>?hard_delete=true"
```

For both soft and hard deletes, EvalHub cleans up associated **Job** and **ConfigMap** Kubernetes resources in the tenant namespace before updating or removing the record. The server returns **204 No Content** on success.

Alternatively, use the CLI.

To cancel a running job with a soft delete:

```
$ evalhub eval cancel <job_id>
```

To permanently delete a job with a hard delete:

```
$ evalhub eval cancel <job_id> --hard
```

Alternatively, use the Python SDK.

To cancel a running job with a soft delete:

```
client.jobs.cancel(job_id)
```

To permanently delete a job with a hard delete:

```
client.jobs.cancel(job_id, hard_delete=True)
```

Verification

1. For a soft delete, verify the job status is **cancelled**:

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>" \
    $EVALHUB_URL/api/v1/evaluations/jobs/<job_id> | jq .status.state
```

Alternatively, use the CLI:

```
$ evalhub eval status <job_id>
```

Alternatively, use the Python SDK:

```
job = client.jobs.get(job_id)
print(job.state)
```

2. For a hard delete, verify the job returns **404 Not Found**:

```
$ curl -s -o /dev/null -w "%{http_code}" \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Tenant: <namespace>" \
  $EVALHUB_URL/api/v1/evaluations/jobs/<job_id>
```

The CLI and Python SDK raise an error when retrieving a hard-deleted job, confirming that the record has been removed.

5.9. EVALHUB BUILT-IN COLLECTIONS

EvalHub includes several built-in collections that group benchmarks from one or more providers into reusable evaluation suites. Each benchmark in a collection can have its own weight, primary score metric, and pass criteria threshold. For more information, see Section 5.1, “Understanding EvalHub”.

Table 5.3. Built-in collections

Collection	Category	Description	Benchmarks
leaderboard-v2	general	Open LLM Leaderboard v2. Comprehensive evaluation suite for general-purpose language models.	**leaderboard_ifeval**, **leaderboard_bbh**, **leaderboard_gpqa**, **leaderboard_mmlu_pro**, **leaderboard_musr**, **leaderboard_math_hard**
safety-and-fairness-v1	safety	Evaluates model safety, bias, and fairness across diverse scenarios.	**truthfulqa_mc1**, **toxigen**, **winogender**, **crows_pairs_english**, **bbq**, **ethics_cm**
toxicity-and-ethical-principles	safety	End-to-end safety assessment covering toxic content generation, tendency to produce false or misleading information, and alignment with ethical principles.	**toxigen**, **truthfulqa_mc1**, **hhh_alignment**

Each built-in collection defines per-benchmark weights and thresholds. For example, the **safety-and-fairness-v1** collection assigns higher weights to **toxigen** and **ethics_cm** (weight 3) than to **winogender** and **crows_pairs_english** (weight 1), which gives these benchmarks greater influence on the overall safety score.

5.10. CREATE A CUSTOM COLLECTION IN EVALHUB

Create a custom collection that groups benchmarks from one or more providers into a reusable evaluation job.

Prerequisites

- You have a running EvalHub instance.

Procedure

1. Create a collection:

```
$ curl -X POST $EVALHUB_URL/api/v1/evaluations/collections \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "X-Tenant: <namespace>" \
  -d '{
    "name": "my-safety-suite",
    "category": "safety",
    "benchmarks": [
      {"provider_id": "lm_evaluation_harness", "benchmark_id": "truthfulqa_mc2"},
      {"provider_id": "garak", "benchmark_id": "owasp_llm_top_10"}
    ]
  }'
```

Example response

```
{
  "resource": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "tenant": "<namespace>",
    "created_at": "2026-04-22T10:00:00Z",
    "owner": "<username>"
  },
  "name": "my-safety-suite",
  "category": "safety",
  "benchmarks": [
    {"provider_id": "lm_evaluation_harness", "id": "truthfulqa_mc2"},
    {"provider_id": "garak", "id": "owasp_llm_top_10"}
  ]
}
```

Alternatively, use the CLI with a YAML spec file:

my-safety-suite.yaml

```
name: my-safety-suite
category: safety
benchmarks:
  - provider_id: lm_evaluation_harness
    benchmark_id: truthfulqa_mc2
  - provider_id: garak
    benchmark_id: owasp_llm_top_10
```

```
$ evalhub collections create --file my-safety-suite.yaml
```

Alternatively, use the Python SDK:

```
collection = client.collections.create({
    "name": "my-safety-suite",
    "category": "safety",
    "benchmarks": [
        {"provider_id": "lm_evaluation_harness", "benchmark_id": "truthfulqa_mc2"},
        {"provider_id": "garak", "benchmark_id": "owasp_llm_top_10"}
    ]
})
```

Verification

- Confirm the collection was created:

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>" \
    $EVALHUB_URL/api/v1/evaluations/collections/<collection_id> | jq .
```

Alternatively, use the CLI:

```
$ evalhub collections describe <collection_id>
```

Alternatively, use the Python SDK:

```
collection = client.collections.get(collection_id)
```

Using a collection in a job

After creating a collection, you can submit evaluation jobs that reference it. The following example shows a job submission using the created collection:

```
$ curl -X POST $EVALHUB_URL/api/v1/evaluations/jobs \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "X-Tenant: <namespace>" \
  -d '{
    "model": {
      "url": "http://my-model.my-namespace.svc.cluster.local:8080/v1",
      "name": "my-model"
    },
    "collection": {
      "id": "<collection_id>"
    }
  }'
```

5.11. CONFIGURE API KEY AUTHENTICATION FOR MODEL ENDPOINTS

Configure EvalHub to authenticate to a model endpoint using an API key stored as a Kubernetes Secret.

Prerequisites

- You have the model endpoint url.

- You have the API key for your model endpoint.

Procedure

1. Create a Secret containing your API key:

   model-auth.yaml

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: model-auth
   type: Opaque
   stringData:
     api-key: "<api-key>"
   ```

2. Apply the Secret to the tenant namespace:

   ```
   $ oc apply -f model-auth.yaml -n <namespace>
   ```

Verification

- Confirm that the Secret was created and contains the expected **api-key** key:

  ```
  $ oc get secret model-auth -n <namespace> -o jsonpath='{.data}' | jq 'keys'
  ```

  The output should include **<api-key>**.

Next steps

When you submit an evaluation job, include an **auth** field in the **model** object to reference the Secret:

Example model configuration with API key authentication

```
"model": {
  "url": "http://my-model.my-namespace.svc.cluster.local:8080/v1",
  "name": "my-model",
  "auth": {
    "secret_ref": "model-auth"
  }
}
```

where:

secret_ref

Specifies the name of the Secret that contains the API key.

- Section 5.6, “Submit an evaluation job”

5.12. AUTHENTICATE MODELS WITH A SERVICEACCOUNT TOKEN

For models served with KServe and protected by kube-rbac-proxy, EvalHub can use automatic **ServiceAccount** token injection.

Procedure

1. Create a RoleBinding granting the job **ServiceAccount** access to the model's InferenceService.

For more information about creating a **ServiceAccount** and RoleBinding for model authentication, see *Making authenticated inference requests* in *Deploying models with distributed inference*.

5.13. USE CUSTOM DATA FROM S3 FOR EVALHUB EVALUATIONS

You can load external test datasets from S3-compatible storage, such as MinIO or Amazon S3, before an evaluation runs. When configured, EvalHub schedules an init container that downloads the data to **/test_data** inside the Job pod. The adapter can then read the files from that path.

NOTE

This feature only applies when EvalHub runs benchmarks as Jobs. It does not apply to local-only evaluation runs.

Prerequisites

- You have an S3-compatible storage endpoint with your test dataset already uploaded to a bucket.
- You have the S3 credentials for your storage endpoint.

Procedure

1. Create a **Secret** containing your S3 credentials:

   my-s3-credentials.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: my-s3-credentials
  namespace: <namespace>
type: Opaque
stringData:
  AWS_ACCESS_KEY_ID: "<your-access-key>"
  AWS_SECRET_ACCESS_KEY: "<your-secret-key>"
  AWS_DEFAULT_REGION: "<your-region>"
  AWS_S3_ENDPOINT: "<your-s3-endpoint>"
```

   where:

   AWS_DEFAULT_REGION

   Specifies the region for your S3-compatible storage, for example **us-east-1**.

   AWS_S3_ENDPOINT

Specifies the endpoint URL for your S3-compatible storage, for example **https://minio.example.com:9000** for MinIO. For Amazon S3, you can omit this field or use the default AWS endpoint.

```
$ oc apply -f my-s3-credentials.yaml
```

2. When you submit an evaluation job, add a **test_data_ref** block to each benchmark that requires external data:

Example S3 test data configuration in a job submission

```
"benchmarks": [
  {
    "provider_id": "lm_evaluation_harness",
    "benchmark_id": "mmlu",
    "test_data_ref": {
      "s3": {
        "bucket": "my-eval-data",
        "key": "datasets/mmlu",
        "secret_ref": "my-s3-credentials"
      }
    }
  }
]
```

where:

s3.bucket

Specifies the S3 bucket name.

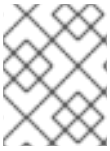
s3.key

Specifies the S3 key prefix for the dataset files.

s3.secret_ref

Specifies the name of the **Secret** containing the S3 credentials.
For the full job submission request, see Section 5.6, “Submit an evaluation job”.

The init container downloads all objects under the specified S3 prefix to **/test_data**, preserving the relative directory structure. The **secret_ref** must reference a **Secret** in the tenant namespace.



NOTE

The expected file format and directory structure of the test data depend on the adapter and benchmark. See the adapter documentation for the required data layout.

Alternatively, use the CLI:

```
$ evalhub eval run \
    --name s3-data-eval \
    --model-url http://my-model.my-namespace.svc.cluster.local:8080/v1 \
    --model-name my-model \
```

```
    --provider lm_evaluation_harness \
    --benchmark mmlu \
    --test-data-s3-bucket my-eval-data \
    --test-data-s3-key datasets/mmlu \
    --test-data-s3-secret my-s3-credentials
```

Alternatively, use the Python SDK:

```
from evalhub.models import (
    JobSubmissionRequest, ModelConfig, BenchmarkConfig,
    TestDataRef, S3TestDataRef
)

job = client.jobs.submit(JobSubmissionRequest(
    name="s3-data-eval",
    model=ModelConfig(
        url="http://my-model.my-namespace.svc.cluster.local:8080/v1",
        name="my-model"
    ),
    benchmarks=[
        BenchmarkConfig(
            id="mmlu",
            provider_id="lm_evaluation_harness",
            test_data_ref=TestDataRef(
                s3=S3TestDataRef(
                    bucket="my-eval-data",
                    key="datasets/mmlu",
                    secret_ref="my-s3-credentials",
                )
            ),
        )
    ],
))
```

Collections also support **test_data_ref** on individual benchmarks, allowing you to define custom data sources as part of a reusable evaluation suite.

Verification

- Confirm that the job completes successfully. If the init container fails to download data from S3, the job transitions to the **failed** state.

  ```
  $ curl -s \
      -H "Authorization: Bearer $TOKEN" \
      -H "X-Tenant: <namespace>" \
      $EVALHUB_URL/api/v1/evaluations/jobs/<job_id> | jq .status.state
  ```

  If the job fails, check the init container logs for download errors:

  ```
  $ oc logs <pod_name> -c init -n <namespace>
  ```

5.14. EXPORT EVALUATION RESULTS TO AN OCI REGISTRY

EvalHub can export evaluation artifacts, such as logs, metrics, and outputs, by pushing artifacts to an Open Container Initiative (OCI) compatible registry for long-term storage and traceability.

Prerequisites

- You have access to an OCI-compatible container registry such as Quay.io.
- You have registry credentials for the OCI registry.

Procedure

1. Create a **kubernetes.io/dockerconfigjson** Secret with your registry credentials:

```
$ oc create secret docker-registry oci-registry-credentials \
    --docker-server=quay.io \
    --docker-username=<username> \
    --docker-password=<password> \
    -n <namespace>
```

2. When you submit an evaluation job, include an **exports** block in the job submission body:

Example OCI export configuration in a job submission

```
"benchmarks": [
  {
    "provider_id": "lm_evaluation_harness",
    "benchmark_id": "mmlu"
  }
],
"exports": {
  "oci": {
    "coordinates": {
      "oci_host": "quay.io",
      "oci_repository": "my-org/eval-results"
    },
    "k8s": {
      "connection": "oci-registry-credentials"
    }
  }
}
```

where:

oci.coordinates.oci_host

Specifies the OCI registry hostname.

oci.coordinates.oci_repository

Specifies the repository path within the registry.

oci.k8s.connection

Specifies the name of the Secret containing the registry credentials.
For the full job submission request, see Section 5.6, “Submit an evaluation job”.

Results artifact from the evaluation frameworks are stored as OCI artifacts with separate layers, allowing selective access to specific outputs.

Verification

1. After the job completes, retrieve the OCI artifact reference from the job results:

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>" \
    $EVALHUB_URL/api/v1/evaluations/jobs/<job_id> | jq '.results.benchmarks[0].artifacts'
```

2. Verify the artifact exists in the registry by using **skopeo**:

```
$ skopeo inspect --creds <username>:<password> docker://quay.io/my-org/eval-results:
<tag>
```

   The tag is in the format **evalhub-<hash>**, where the hash is derived from the job ID, provider, and benchmark. You can find the full OCI reference, including the tag, in the job results.

5.15. CONFIGURE MLFLOW EXPERIMENT TRACKING FOR EVALUATION JOBS

When MLflow is configured for EvalHub, you can associate evaluation jobs with designated MLflow experiments. EvalHub automatically logs benchmark metrics as MLflow runs within the experiment.

Prerequisites

- You have a running MLflow instance accessible from the EvalHub deployment.
- You have configured the MLflow tracking URI in the EvalHub configuration. See Section 5.21, “EvalHub configuration reference” for details.

Procedure

1. When you submit an evaluation job, include an **experiment** block in the job submission body:

   #### Example experiment configuration in a job submission

```
"benchmarks": [
  {
    "provider_id": "lm_evaluation_harness",
    "benchmark_id": "mmlu"
  }
],
"experiment": {
  "name": "my-model-v2-eval"
}
```

   For the full job submission request, see Section 5.6, “Submit an evaluation job”.

When using the CLI, include the **experiment** field in your YAML config file:

Example experiment fragment in a YAML config file

```
experiment:
  name: my-model-v2-eval
```

```
$ evalhub eval run --config eval-with-mlflow.yaml
```

+ For the full YAML config file structure, see Section 5.6, “Submit an evaluation job”.

When using the Python SDK, pass an **ExperimentConfig** to the **JobSubmissionRequest**:

```
from evalhub.models import ExperimentConfig

experiment=ExperimentConfig(name="my-model-v2-eval")
```

+ For the full **JobSubmissionRequest**, see Section 5.6, “Submit an evaluation job”.

Verification

When the job completes, the **results** section includes an **mlflow_experiment_url** linking to the experiment in the MLflow UI:

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>" \
    $EVALHUB_URL/api/v1/evaluations/jobs/<job_id> | jq .results.mlflow_experiment_url
```

Example output

```
"https://mlflow.example.com/#/experiments/42"
```

Alternatively, use the CLI. The **evalhub eval results** command automatically displays the MLflow experiment URL when available:

```
$ evalhub eval results <job_id>
```

Alternatively, use the Python SDK:

```
job = client.jobs.get(job_id)
print(job.results.mlflow_experiment_url)
```

5.16. ADD A CUSTOM PROVIDER BY USING THE API

Register a custom provider by using the REST API. A provider definition includes a name, a container image for the adapter runtime, and a list of benchmarks. For more information about adapters, see Section 5.1, “Understanding EvalHub”.

Prerequisites

- You have a running EvalHub instance.
- You have a container image for your custom adapter packaged as a UBI9 image.

Procedure

1. Register the custom provider:

```
$ curl -X POST $EVALHUB_URL/api/v1/evaluations/providers \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "X-Tenant: <namespace>" \
  -d '{
    "name": "my-custom-provider",
    "title": "My Custom Provider",
    "description": "Custom evaluation framework for domain-specific benchmarks.",
    "benchmarks": [
      {
        "id": "domain_accuracy",
        "name": "Domain Accuracy",
        "category": "general",
        "metrics": ["accuracy", "f1"],
        "primary_score": {
          "metric": "accuracy",
          "lower_is_better": false
        },
        "pass_criteria": {
          "threshold": 0.8
        }
      }
    ],
    "runtime": {
      "k8s": {
        "image": "quay.io/my-org/my-adapter:latest",
        "cpu_request": "500m",
        "memory_request": "512Mi",
        "cpu_limit": "2000m",
        "memory_limit": "4Gi"
      }
    }
  }'
```

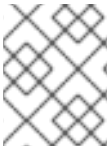
Example response

```
{
  "resource": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "tenant": "<namespace>",
    "created_at": "2026-04-22T10:00:00Z",
    "owner": "<username>"
  },
  "name": "my-custom-provider",
  "title": "My Custom Provider",
  "description": "Custom evaluation framework for domain-specific benchmarks.",
  "benchmarks": [
    {
      "id": "domain_accuracy",
      "name": "Domain Accuracy",
      "category": "general",
      "metrics": ["accuracy", "f1"],
      "primary_score": { "metric": "accuracy", "lower_is_better": false },
      "pass_criteria": { "threshold": 0.8 }
    }
```

```
  ],
  "runtime": {
    "k8s": {
      "image": "quay.io/my-org/my-adapter:latest",
      "cpu_request": "500m",
      "memory_request": "512Mi",
      "cpu_limit": "2000m",
      "memory_limit": "4Gi"
    }
  }
}
```

The **runtime.k8s** section specifies the container image and resource requests for the adapter pod. Each benchmark must declare an **id**, **name**, and **category**. The optional **primary_score** and **pass_criteria** fields set default thresholds for the benchmark.

User-created providers can be updated and deleted through the API. Built-in providers with **owner: system** are read-only.



NOTE

The Python SDK and CLI do not support creating providers. Use the REST API to register custom providers.

Verification

- Confirm the provider was registered by retrieving it with the ID from the response:

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>" \
    $EVALHUB_URL/api/v1/evaluations/providers/<provider_id> | jq .name
```

The output should return **"my-custom-provider"**.

Alternatively, use the CLI:

```
$ evalhub providers describe <provider_id>
```

Alternatively, use the Python SDK:

```
provider = client.providers.get(provider_id)
print(provider.name)
```

5.17. ADD A CUSTOM PROVIDER BY USING A CONFIGMAP

Add providers at the operator level by creating a **ConfigMap** in the operator namespace with the appropriate labels. The TrustyAI Operator discovers **ConfigMap**(s) by label and mounts them into the EvalHub deployment automatically. Providers registered this way are system-owned, read-only, and available to all tenants. To register a tenant-scoped provider that can be updated or deleted, use the REST API instead. See Section 5.16, “Add a custom provider by using the API” .

Prerequisites

- You have a running EvalHub deployment.

- You have a container image for your custom adapter. See Section 5.19, "Write a custom evaluation adapter".
- You have cluster administrator privileges or permissions to create **ConfigMap** resources in the operator namespace.
- You have permissions to edit the EvalHub custom resource.

Procedure

1. Create a **ConfigMap** in the EvalHub custom resource namespace with the provider definition:

   evalhub-provider-my-custom-provider.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: evalhub-provider-my-custom-provider
  namespace: <evalhub-namespace>
  labels:
    trustyai.opendatahub.io/evalhub-provider-type: system
    trustyai.opendatahub.io/evalhub-provider-name: my-custom-provider
data:
  my-custom-provider.yaml: |
    id: my-custom-provider
    name: My Custom Provider
    description: Custom evaluation framework for domain-specific benchmarks.
    runtime:
      k8s:
        image: quay.io/my-org/my-adapter:latest
        cpu_request: "500m"
        memory_request: "512Mi"
        cpu_limit: "2000m"
        memory_limit: "4Gi"
    benchmarks:
      - id: domain_accuracy
        name: Domain Accuracy
        category: general
        metrics:
          - accuracy
          - f1
        primary_score:
          metric: accuracy
          lower_is_better: false
        pass_criteria:
          threshold: 0.8
```

```
$ oc apply -f evalhub-provider-my-custom-provider.yaml
```

2. Reference the provider name in your EvalHub custom resource by adding it to the **spec.providers** list:

   Example spec.providers fragment

```
spec:
  providers:
    - lm_evaluation_harness
    - garak
    - my-custom-provider
```

For the full EvalHub custom resource structure, see Section 5.3, “Deploy EvalHub with the TrustyAI Operator”.

The operator copies the **ConfigMap** to the instance namespace and mounts it as a projected volume at **/etc/evalhub/config/providers**. The EvalHub server loads all provider YAML files from this directory at startup.

Verification

1. Confirm that the **ConfigMap** was created:

```
$ oc get configmap evalhub-provider-my-custom-provider -n <evalhub-namespace>
```

2. Check that the EvalHub deployment has restarted and is ready:

```
$ oc get pods -l app=eval-hub -n <evalhub-namespace>
```

3. Confirm the custom provider is loaded:

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>" \
    $EVALHUB_URL/api/v1/evaluations/providers/my-custom-provider | jq .name
```

The output should return **"My Custom Provider"**.

5.18. ADD A COLLECTION BY USING A CONFIGMAP

Add collections at the operator level by creating a **ConfigMap** in the operator namespace with the appropriate labels. The TrustyAI Operator discovers **ConfigMap**(s) by label and mounts them into the EvalHub deployment automatically. Collections registered this way are system-owned, read-only, and available to all tenants. To create a tenant-scoped collection that can be updated or deleted, use the REST API instead. See Section 5.10, “Create a custom collection in EvalHub” .

Prerequisites

- You have a running EvalHub deployment.
- You have cluster administrator privileges or permissions to create **ConfigMap** resources in the operator namespace.
- You have permissions to edit the EvalHub custom resource.
- You know which provider-benchmark pairs you want to include in the collection. See Section 5.5, “List EvalHub providers and benchmarks” .

Procedure

1. Create a **ConfigMap** in the EvalHub custom resource namespace with the collection definition:

evalhub-collection-my-eval-suite.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: evalhub-collection-my-eval-suite
  namespace: <evalhub-namespace>
  labels:
    trustyai.opendatahub.io/evalhub-collection-type: system
    trustyai.opendatahub.io/evalhub-collection-name: my-eval-suite
data:
  my-eval-suite.yaml: |
    id: my-eval-suite
    name: My Evaluation Suite
    category: general
    description: Custom evaluation suite for internal model validation.
    pass_criteria:
      threshold: 0.7
    benchmarks:
      - id: mmlu
        provider_id: lm_evaluation_harness
        weight: 2
        primary_score:
          metric: acc_norm
          lower_is_better: false
        pass_criteria:
          threshold: 0.6
      - id: hellaswag
        provider_id: lm_evaluation_harness
        weight: 1
        primary_score:
          metric: acc_norm
          lower_is_better: false
        pass_criteria:
          threshold: 0.7
```

```
$ oc apply -f evalhub-collection-my-eval-suite.yaml
```

2. Reference the collection in your EvalHub custom resource by adding the collection name to the **spec.collections** list:

Example spec.collections fragment

```
spec:
  collections:
    - leaderboard-v2
    - safety-and-fairness-v1
    - my-eval-suite
```

For the full EvalHub custom resource structure, see Section 5.3, “Deploy EvalHub with the TrustyAI Operator”.

The operator mounts collection **ConfigMap**(s) at **/etc/evalhub/config/collections**.

Verification

1. Confirm that the **ConfigMap** was created:

```
$ oc get configmap evalhub-collection-my-eval-suite -n <evalhub-namespace>
```

2. Check that the EvalHub deployment has restarted and is ready:

```
$ oc get pods -l app=eval-hub -n <evalhub-namespace>
```

3. List collections and confirm the custom collection appears:

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "X-Tenant: <namespace>" \
    $EVALHUB_URL/api/v1/evaluations/collections/my-eval-suite | jq .name
```

The output should return **"My Evaluation Suite"**.

5.19. WRITE A CUSTOM EVALUATION ADAPTER

An adapter translates EvalHub job requests into evaluation framework-specific commands. To write a custom adapter, install the EvalHub SDK with adapter dependencies and implement a single method.

Prerequisites

- You have Python 3.11 or later installed.
- You have an evaluation framework that you want to integrate with EvalHub.
- You have **podman** or another container build tool installed to package the adapter as a container image.

Procedure

1. Install the EvalHub SDK with the adapter extra:

```
$ pip install "eval-hub-sdk[adapter]"
```

2. Create a class that extends **FrameworkAdapter** and implements **run_benchmark_job**:

```
from evalhub.adapter import FrameworkAdapter
from evalhub.models import JobSpec, JobCallbacks, JobResults, JobStatusUpdate,
JobPhase

class MyAdapter(FrameworkAdapter):
    def run_benchmark_job(self, config: JobSpec, callbacks: JobCallbacks) -> JobResults:
        callbacks.report_status(JobStatusUpdate(
            phase=JobPhase.RUNNING_EVALUATION,
            message="Running evaluation"
        ))

        # Replace with your framework's evaluation function
        scores = run_my_framework(
            model_url=config.model.url,
            benchmark=config.benchmark_id,
```

```
        parameters=config.parameters
    )

    return JobResults(
        id=config.id,
        benchmark_id=config.benchmark_id,
        benchmark_index=config.benchmark_index,
        model_name=config.model.name,
        results=scores,
        num_examples_evaluated=len(scores),
        duration_seconds=self._get_duration()  # Implement to return elapsed seconds
    )
```

The framework handles loading the job specification from the mounted **ConfigMap**, authenticating with the sidecar proxy container that communicates with the EvalHub server, and reporting results. Your adapter only needs to run the evaluation and return the results. For more information about the adapter and sidecar architecture, see Section 5.2, "EvalHub architecture overview".

3. Package your adapter as a Red Hat Universal Base Image 9 (UBI9) container image. Create a **Containerfile** in your adapter directory:

   Containerfile

   ```
   FROM registry.access.redhat.com/ubi9/python-312

   WORKDIR /app

   COPY requirements.txt .
   RUN pip install --no-cache-dir -r requirements.txt

   COPY main.py /app/main.py

   ENTRYPOINT ["python", "main.py"]
   ```

   Build the image:

   ```
   $ podman build -t quay.io/my-org/my-adapter:latest .
   ```

   Push the image to a container registry:

   ```
   $ podman push quay.io/my-org/my-adapter:latest
   ```

4. Reference the image in the provider's **runtime.k8s.image** field when registering the provider. See Section 5.16, "Add a custom provider by using the API" .

The following tables describe the **JobSpec** and **JobCallbacks** interfaces available to your adapter.

Table 5.4. JobSpec fields

Field	Description
id	Unique job identifier.

Field	Description
provider_id	Identifier of the provider that the benchmark belongs to.
benchmark_id	Identifier of the benchmark to evaluate.
benchmark_index	Index of this benchmark within the job.
model	Model configuration, including **url** and **name**.
parameters	Benchmark-specific parameters, for example **num_fewshot** or **limit**.
num_examples	The number of examples to evaluate. When set to **None**, the adapter evaluates all examples.
exports	Optional OCI artifact export specification.

Table 5.5. JobCallbacks methods

Method	Purpose
report_status(update)	Sends progress updates including the phase, message, and completed/total steps.
create_oci_artifact(spec)	Pushes evaluation artifacts to an OCI registry.
report_results(results)	Reports the final results to the EvalHub server. This method is called automatically if you return **JobResults**.

5.20. EVALHUB API ENDPOINTS

All endpoints use the path prefix **/api/v1**. The OpenAPI 3.1.0 specification is available at **/openapi.yaml** and interactive documentation is available at **/docs**.

5.20.1. Evaluation job endpoints

Table 5.6. Evaluation job endpoints

Endpoint	Method	Description
/api/v1/evaluations/jobs	POST	Create and submit an evaluation job. Returns **202 Accepted**.
/api/v1/evaluations/jobs	GET	List evaluation jobs with pagination and filtering.

Endpoint	Method	Description
/api/v1/evaluations/jobs/\{id}	GET	Get a specific evaluation job with current status and results.
/api/v1/evaluations/jobs/\{id}	DELETE	Cancel or hard-delete a job. Use **? hard_delete=true** for permanent removal.
/api/v1/evaluations/jobs/\{id}/events	POST	Submit job status events from the adapter runtime.

Table 5.7. Evaluation job states

State	Description
pending	The job is created and awaiting execution.
running	The evaluation is actively running.
completed	All benchmarks completed successfully.
failed	The evaluation encountered a fatal error.
cancelled	The user canceled the job.
partially_failed	Some benchmarks succeed and others failed.

5.20.2. Provider endpoints

Table 5.8. Provider endpoints

Endpoint	Method	Description
/api/v1/evaluations/providers	POST	Create a custom provider.
/api/v1/evaluations/providers	GET	List providers. Use **?benchmarks=true** to include benchmarks.
/api/v1/evaluations/providers/\{id}	GET	Get a provider with all its benchmarks.
/api/v1/evaluations/providers/\{id}	PUT	Replace a provider.

Endpoint	Method	Description
/api/v1/evaluations/providers/{id}	PATCH	Patch a provider with JSON Patch operations.
/api/v1/evaluations/providers/{id}	DELETE	Delete a provider.

Table 5.9. Built-in providers

Provider	Benchmarks	Description
lm_evaluation_harness	167	General-purpose LLM evaluation: MMLU, HellaSwag, ARC, TruthfulQA, GSM8K, and more across 12 categories.
garak	8	Security vulnerability scanning: OWASP LLM Top 10, AVID taxonomy, CWE.
guidellm	7	Guidance language model evaluation.
lighteval	24	Lightweight evaluation framework.

5.20.3. Collection endpoints

Table 5.10. Collection endpoints

Endpoint	Method	Description
/api/v1/evaluations/collections	POST	Create a benchmark collection.
/api/v1/evaluations/collections	GET	List collections with filtering.
/api/v1/evaluations/collections/{id}	GET	Get a collection with all benchmark references.
/api/v1/evaluations/collections/{id}	PUT	Replace a collection.
/api/v1/evaluations/collections/{id}	PATCH	Patch a collection with JSON Patch operations.
/api/v1/evaluations/collections/{id}	DELETE	Delete a collection.

5.20.4. Health and observability endpoints

Table 5.11. Health and observability endpoints

Endpoint	Method	Description
/api/v1/health	GET	Health check with status, timestamp, and build information.
/metrics	GET	Prometheus metrics endpoint when enabled.
/openapi.yaml	GET	OpenAPI 3.1.0 specification in YAML or JSON based on Accept header.
/docs	GET	Interactive Swagger UI documentation.

5.21. EVALHUB CONFIGURATION REFERENCE

Configuration applies to the EvalHub server component. EvalHub is configured by using **config/config.yaml** and environment variables. Environment variables take precedence over the configuration file.

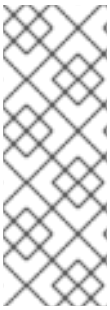
When deploying EvalHub with the TrustyAI Operator, the operator generates the **config.yaml** automatically from the EvalHub custom resource and environment variables defined in the **spec.env** field. You do not need to create or edit **config.yaml** directly. For information about configuring the EvalHub custom resource, see Section 5.3, “Deploy EvalHub with the TrustyAI Operator” .

5.21.1. Service configuration

Table 5.12. Service parameters

Parameter	Environment variable	Default	Description
service.port	**PORT**	**8080**	The port that the API server listens on.
service.host	**API_HOST**	**127.0.0.1**	The address that the API server binds to.
service.tls_cert_file	**TLS_CERT_FILE**	–	Path to the TLS certificate file.
service.tls_key_file	**TLS_KEY_FILE**	–	Path to the TLS private key file.
service.disable_auth	–	**false**	Disables authentication and authorization. Setting this to **true** allows unauthenticated access to all endpoints. Do not enable this in production environments.

5.21.2. Database configuration



NOTE

When deploying EvalHub with the TrustyAI Operator, you must set **spec.database.type** in the EvalHub custom resource to either **postgresql** or **sqlite**. The operator generates the corresponding configuration automatically. The **postgresql** option sets the driver to **pgx** and injects the connection URL from a Kubernetes Secret. The **sqlite** option sets the driver to **sqlite** with an in-memory database. Data is not persisted across restarts with **sqlite**. Use **postgresql** for production deployments.

The following table describes the parameters available in the EvalHub **config/config.yaml** configuration file.

Table 5.13. Database parameters

Parameter	Environment variable	Default	Description
database.driver	–	**sqlite**	The storage driver. Supported values: **sqlite**, **pgx**. The default **sqlite** option uses an in-memory database and data is not persisted across restarts. Use **pgx** with PostgreSQL for production deployments.
database.url	**DB_URL**	**file::eval_hub:?mode=memory&cache=shared**	The database connection string. The default value is a SQLite in-memory URI, which stores all data in memory and does not persist across restarts. For PostgreSQL, use the format **postgres://user:password@host:5432/eval_hub**. Store the connection string in a Kubernetes Secret rather than inline to avoid exposing credentials. For instructions, see Section 5.3, "Deploy EvalHub with the TrustyAI Operator".

5.21.3. MLflow configuration

Table 5.14. MLflow parameters

Parameter	Environment variable	Default	Description
mlflow.tracking_uri	**MLFLOW_TRACKING_URI**	–	The URL of the MLflow tracking server. Setting this parameter enables MLflow integration. When set, evaluation results are logged to MLflow. Without this parameter, MLflow tracking is disabled.
mlflow.ca_cert_path	**MLFLOW_CA_CERT_PATH**	–	The path to a TLS CA certificate file for verifying the MLflow server's certificate.

Parameter	Environment variable	Default	Description
mlflow.insecure_skip_verify	**MLFLOW_INSECURE_SKIP_VERIFY**	**false**	If **true**, skips TLS certificate verification when connecting to MLflow. Use this option only for testing with self-signed certificates. Do not enable this in production environments.
mlflow.token_path	**MLFLOW_TOKEN_PATH**	–	The path to a file containing an authentication token for the MLflow server. The token is sent as a Bearer token in the **Authorization** header. The default path is **/var/run/secrets/mlflow/token**, which is a projected **ServiceAccount** token.
mlflow.workspace	**MLFLOW_WORKSPACE**	–	The MLflow workspace or experiment namespace.

5.21.4. OpenTelemetry configuration

When deploying with the TrustyAI Operator, include the **otel** field in the EvalHub custom resource to enable OpenTelemetry. The presence of the **otel** field in the CR enables OpenTelemetry automatically.

Table 5.15. OpenTelemetry parameters available in the EvalHub custom resource

CR field	Default	Description
otel.exporterType	**otlp-grpc**	The exporter type. Supported values: **otlp-grpc**, **otlp-http**, **stdout**.
otel.exporterEndpoint	–	The endpoint for the OTLP exporter, for example **localhost:4317** for gRPC.
otel.exporterInsecure	**false**	If **true**, disables TLS for the OTLP exporter connection. Do not enable this in production environments.
otel.samplingRatio	**1.0**	Trace sampling ratio as a value between **0** and **1**. For example, **0.5** samples 50% of traces.

5.22. EVALHUB MULTI-TENANCY AND RBAC

EvalHub supports namespace-based multi-tenancy, where each Kubernetes namespace represents a tenant. EvalHub enforces isolation at multiple layers, including authentication, authorization, data access, and job execution.

EvalHub enforces isolation at the following layers:

- **Authentication** – EvalHub uses the Kubernetes **TokenReview** API to validate bearer tokens in incoming requests.

- **Authorization** – **SubjectAccessReview** (SAR) checks verify that the caller has permission to perform the requested operation on EvalHub virtual resources in the target namespace. Virtual resources are logical resource names that EvalHub defines for RBAC purposes under the **trustyai.opendatahub.io** API group. They do not correspond to Kubernetes custom resource definitions. The virtual resources are **evaluations**, **collections**, **providers**, and **status-events**. For the full list of verbs, see Section 5.25, "EvalHub roles reference" .
- **Data isolation** – EvalHub scopes all database queries by **tenant_id** to prevent cross-tenant data access.
- **Job execution** – EvalHub creates Job resources in the tenant's namespace.

The **X-Tenant** request header determines the target tenant namespace. The **X-User** header identifies the authenticated user.

5.23. SET UP A TENANT NAMESPACE

Register a namespace as an EvalHub tenant so that users, programmatic clients, and agents can submit evaluation jobs in that namespace.

Prerequisites

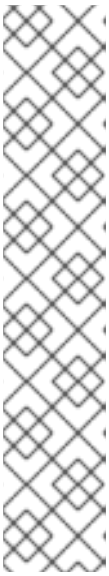
- You have cluster administrator privileges.
- You have a running EvalHub instance.
- You have a namespace to use as a tenant.

Procedure

1. Add the tenant label to the namespace:

```
$ oc label namespace <namespace> evalhub.trustyai.opendatahub.io/tenant=
```

The label value is intentionally empty. The TrustyAI Operator checks for the presence of the label, not its value.



NOTE

Use a dedicated namespace for EvalHub rather than **redhat-ods-applications**, as described in Section 5.3, "Deploy EvalHub with the TrustyAI Operator" . The **redhat-ods-applications** namespace has **NetworkPolicy** resources that restrict cross-namespace traffic, which requires additional labeling on tenant namespaces. If EvalHub is deployed in **redhat-ods-applications**, label each tenant namespace to allow the evaluation **Job** sidecar to communicate with the EvalHub server:

```
$ oc label namespace <namespace> opendatahub.io/generated-
namespace=true
```

Review the **NetworkPolicy** resources with **oc get networkpolicy -n <evalhub-server-namespace>** to determine any additional requirements.

The TrustyAI Operator watches for this label and automatically provisions the following resources in the labeled namespace:

- A job **ServiceAccount** used by evaluation **Job** pods as their identity.
- A **Role** and **RoleBinding** granting the job **ServiceAccount** permission to create **status-events** for reporting job progress.
- A **RoleBinding** granting the EvalHub API **ServiceAccount** permission to create and delete **Job** resources in the tenant namespace.
- A **RoleBinding** granting the EvalHub API **ServiceAccount** permission to manage **ConfigMap** resources used to mount job specifications into **Job** pods.
- A **RoleBinding** granting the job **ServiceAccount** access to MLflow resources when MLflow is configured.
- A service CA **ConfigMap** with the cluster CA bundle injected by OpenShift, so that **Job** pods can make HTTPS requests to the EvalHub API.

When the tenant label is removed from a namespace, the controller cleans up all provisioned resources automatically.

Verification

1. Confirm that the tenant label is set on the namespace:

```
$ oc get namespace <namespace> --show-labels | grep evalhub
```

2. Confirm that the operator provisioned the expected resources in the tenant namespace:

```
$ oc get serviceaccount,rolebinding,configmap -n <namespace> | grep evalhub
```

   The output should include a **ServiceAccount**, **RoleBinding** resources, and a service CA **ConfigMap** created by the operator.

5.24. GRANT ACCESS TO EVALHUB

Grant tenant users access to EvalHub by creating a **Role** and **RoleBinding** in the tenant namespace. EvalHub supports three types of principals.

Prerequisites

- You have permissions to create **Role** and **RoleBinding** resources in the tenant namespace.
- You have impersonation privileges to verify access with **oc auth can-i --as**.
- You have set up the target namespace as an EvalHub tenant.
- You have identified which virtual resources and verbs to grant. See Section 5.25, “EvalHub roles reference” for available resources.

Procedure

Select the type of principal that matches your use case.

Table 5.16. Principal types

Principal type	Token source	Use case
ServiceAccount	Mounted pod token or long-lived token	Automation, CI/CD pipelines, agents using Model Context Protocol (MCP)
OpenShift User	**oc whoami -t**	Interactive use
OpenShift Group	User token with group membership	Team-based access

1. Create a **Role** in the tenant namespace that grants access to the required EvalHub virtual resources:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: evalhub-evaluator
  namespace: <namespace>
rules:
  - apiGroups: ["trustyai.opendatahub.io"]
    resources: ["evaluations", "collections", "providers"]
    verbs: ["get", "list", "create", "update", "delete"]
  - apiGroups: ["mlflow.kubeflow.org"]
    resources: ["experiments"]
    verbs: ["create", "get"]
```

```
$ oc apply -f evalhub-evaluator-role.yaml
```

2. Create a **RoleBinding** to bind the principal to the **Role**.

   - To grant access to a ServiceAccount:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: my-sa-evalhub-access
  namespace: <namespace>
subjects:
  - kind: ServiceAccount
    name: my-sa
    namespace: <namespace>
roleRef:
  kind: Role
  name: evalhub-evaluator
  apiGroup: rbac.authorization.k8s.io
```

```
$ oc apply -f my-sa-evalhub-access.yaml
```

   To obtain a bearer token for a **ServiceAccount**, run the following command:

```
$ export TOKEN=$(oc create token my-sa -n <namespace> --duration=1h)
```

- To grant access to a User:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: user-evalhub-access
  namespace: <namespace>
subjects:
  - kind: User
    name: <username>
roleRef:
  kind: Role
  name: evalhub-evaluator
  apiGroup: rbac.authorization.k8s.io
```

```
$ oc apply -f user-evalhub-access.yaml
```

To obtain a bearer token for an OpenShift User, log in as the user and run the following command:

```
$ export TOKEN=$(oc whoami -t)
```

- To grant access to a Group:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: team-evalhub-access
  namespace: <namespace>
subjects:
  - kind: Group
    name: evalhub-users
roleRef:
  kind: Role
  name: evalhub-evaluator
  apiGroup: rbac.authorization.k8s.io
```

```
$ oc apply -f team-evalhub-access.yaml
```

To obtain a bearer token for a Group member, log in as a user who belongs to the group and run the following command:

```
$ export TOKEN=$(oc whoami -t)
```

Verification

Verify that the principal has the expected permissions on the EvalHub virtual resources by using **oc auth can-i**.

- For a **ServiceAccount**:

```
$ oc auth can-i create evaluations.trustyai.opendatahub.io \
    -n <namespace> \
    --as=system:serviceaccount:<namespace>:my-sa
```

- For an OpenShift User:

```
$ oc auth can-i create evaluations.trustyai.opendatahub.io \
    -n <namespace> \
    --as=<username>
```

- For an OpenShift Group:

```
$ oc auth can-i create evaluations.trustyai.opendatahub.io \
    -n <namespace> \
    --as=<username> --as-group=evalhub-users
```

Each command should return **yes**.

5.25. EVALHUB ROLES REFERENCE

EvalHub uses virtual Kubernetes resources for tenant authorization. These resources do not correspond to actual Kubernetes API resources. EvalHub performs SubjectAccessReview (SAR) checks against these resources in the tenant namespace specified by the **X-Tenant** header.

To authorize tenant users, create a Role in the tenant namespace granting the required verbs on these virtual resources. For instructions, see Section 5.24, “Grant access to EvalHub” .

Table 5.17. Virtual resources for tenant authorization

API group	Resource	Verbs	Description
trustyai.opendatahub.io	**evaluations**	**get**, **list**, **create**, **update**, **delete**	Submit, view, update, and delete evaluation jobs.
trustyai.opendatahub.io	**collections**	**get**, **list**, **create**, **update**, **delete**	Create, view, update, and delete benchmark collections.
trustyai.opendatahub.io	**providers**	**get**, **list**, **create**, **update**, **delete**	Create, view, update, and delete evaluation providers.
trustyai.opendatahub.io	**status-events**	**create**	Report job progress. Used by operator-provisioned job ServiceAccounts, not by tenant users.
mlflow.kubeflow.org	**experiments**	**create**, **get**	Create and access MLflow experiments for result tracking.

5.26. ADDITIONAL RESOURCES

The following resources provide additional information about EvalHub.

- EvalHub documentation site
- Server API reference – REST API endpoints and configuration
- Python SDK reference – Client library documentation
- CLI reference – Command-line interface guide
- Architecture guide – Adapter pattern and adapter development
- Multi-tenancy guide – Detailed RBAC and tenant configuration